[Experiment, Analysis, and Benchmark] Adaptive Retrieval Versus a Coin Flip: Cost-Matched Baselines Change the Verdict

Yasir Saleem, Xiaofeng Ding, Ghalib Nadeem
School of Computer Science and Technology
Huazhong University of Science and Technology
Wuhan, China
yasir_saleem@hust.edu.cn, xfding@hust.edu.cn, ghalibnadeem@gmail.com

***Abstract*—Adaptive retrieval policies decide for each question whether to retrieve, with which retriever and how many passages. Papers report that these policies beat fixed retrieval settings, but most measure the gain against one fixed setting. We show that this choice of baseline can decide the verdict. A random mix of two fixed plans reaches any cost between the two plans and uses no information about the question; the best such mix at the policy's own mean latency, chosen on training folds, is the stronger baseline. We built FunnelPlan, a benchmark that answers 2,000 questions from four datasets with up to 33 fixed plans and five open readers, and stores a graded answer and a measured cost for each pair. On the same answers and under one test, five primary policies beat Always RAG and Never RAG in 23 of 27 pooled tests, the best single plan at the same cost in 5 to 7, and the cost-matched mix in none. Adaptive RAG run as published beats Always RAG by 0.070 to 0.091, but a random mix of its own rungs at its cost does as well, and one fixed plan at under a fifth of its latency scores 0.023 to 0.061 higher. No policy that picks one plan per question beats the mix under four hardware-free costs, three grading rules or a hull matched at the 95th or 99th percentile latency; 53 of 54 upper bounds exclude a gain of 0.03 under the per-question test, the corrected test and a refit bootstrap alike, and the closest test stays below 0.04 on labels corrected by two human audits. On a mixed workload, routers that pay for their query embedding beat the best global mix under some tests with two-hop plans, but none meets our decision rule. IRCoT beats the mix on 2Wiki, but chains that run its own reasoning for a fixed number of steps match it, so its stopping rule adds nothing. These results cover open readers of 7B to 32B parameters served one question at a time and short-answer English questions.**

***Index Terms*—retrieval-augmented generation, adaptive retrieval, query routing, benchmark, cost-aware evaluation**

I. Introduction

Retrieval-augmented generation (RAG) retrieves passages and gives them to a language model, the reader, which writes the answer [1]. Each question can be answered with a different retrieval plan: no retrieval, a choice of retriever and depth, or a second round of retrieval. Deeper and stronger plans cost more latency and more tokens. Adaptive retrieval policies therefore choose a plan per question. Some retrieve only when the model seems not to know the answer [2], [3] or when its confidence drops [4]. Others route by predicted question complexity [5] or set the depth from the retrieval scores [6]. Data management venues now publish such planners as well [7]–[9].

Most of these papers measure gains against one fixed setting, such as Always RAG at one depth, or against other adaptive methods [5], [6], [10]. That comparison misses a cheap competitor. Run plan a on some questions and plan b on the rest, chosen at random. This mix reaches any cost between the two plans, and its success lies on the straight line between them [11]. It uses no information about the question. A policy that cannot beat this line gives nothing that a coin flip between two fixed plans would not give at the same cost. LLM routing already uses this baseline: Hybrid LLM and RouteLLM compare their routers with random routing between two models at the same cost [12], [13]. Adaptive retrieval papers rarely do. Weak baselines have inflated reported progress before, in information retrieval [14], [15] and in recommendation [16]. In data management, careful benchmarks found that learned query optimizers did not consistently beat PostgreSQL [17], and that a simple adaptive baseline matched or beat them on many workloads [18].

This paper measures how much the baseline and the protocol change the verdict on adaptive retrieval, once policies are compared at equal measured cost with the best mix of fixed plans. Answering this needs four things that existing benchmarks do not combine (Table I). The first is a plan matrix: every question answered by every plan, so that any policy and any mix can be scored on the same answers. The second is measured cost. The third is a protocol that chooses the settings of both the policy and the mix on the same training data. The fourth is grading checked by humans, with enough questions to detect a small gain.

We built FunnelPlan (Fig. 1). It answers 2,000 questions from four QA datasets with up to 33 fixed plans and five open readers from 7B to 32B parameters. It stores about 300,000 graded answers, each with its measured latency, tokens and calls. On this matrix we replay 15 adaptive policies, Adaptive RAG as published and the multi-step method IRCoT [22], and compare each with the fixed plan hull at its own mean cost. We carry the random routing baseline over to retrieval plans with measured costs, choose it on the same training folds as

TABLE I: RAG benchmarks compared on design features. Pol.: adaptive retrieval policies compared. Hull: compared with a mix of fixed plans at equal cost. Rd.: readers in the main results. Lat.: measured latency per question (rel.: only relative to a one-step plan). Stat.: intervals or significance tests. Hum.: human audit of answer grading.

Benchmark	Pol.	Hull	Rd.	Lat.	Stat.	Hum.
Adaptive RAG [5]	✓		3	rel.		
Moskvoretskii et al. [10]	✓		1		✓	
FlashRAG [19]	✓		2			
RAGPerf [20]			5	✓		
XRAG [21]			3			
FunnelPlan (ours)	✓	✓	5	✓	✓	✓

the policy, and report how much power each test has. The baseline decides the verdict. Under one pooled test, the same primary policies beat Always RAG and Never RAG in 23 of 27 tests, the best single plan at the same cost in 5 to 7, and the cost-matched mix in none; their wins over the best single plan match what random mixing gains. Adaptive RAG run as published beats Always RAG by 0.070 to 0.091, yet it does no better than a random mix of its own rungs at its cost. For the policies we evaluate, which pick one plan per question, the remaining evidence is narrow: with hulls and policies fitted per dataset, no tested policy beats the mix once it pays for its own signals; on a mixed workload, routers gain mostly by recognising the dataset and none meets our decision rule; and IRCoT's wins on 2Wiki do not depend on its stopping rule. We make three contributions.[1]

A cost-matched protocol, and how much it changes the verdict. Policies are compared with the fixed plan hull at equal mean cost on held-out folds, over five folds and five seeds, with Holm's correction, power and margin tests, and a rule that results must hold with and without two-hop plans. With the test held fixed, the comparator alone takes 23 wins of 27 to none, and wins over the best single plan, up to $+0.080$, are what random mixing gains. Adaptive RAG as published shows the same reversal: its classifier adds nothing over a coin flip between its own rungs.

A plan matrix benchmark. FunnelPlan covers six retrievers at five depths, closed-book answers and fixed two-hop plans, with grades from a strict rule and a local judge audited by three human raters. Any new policy that chooses among these plans can be scored without new reader calls; signals it computes itself are measured and charged.

Evidence on the evaluated policies. None of the 27 primary tests is positive in either plan set; 53 of the 54 one-sided upper bounds exclude a gain of 0.03 under the per-question test, the corrected test and a five-seed refit bootstrap, and on audited labels the confidence cascade for Qwen2.5 7B stays below 0.04 but not below 0.03. Charged for its own query embedding, the best router gains at most $+0.014$. An oracle that picks the right plan per question would gain $+0.098$, but our planners pick the plan that answers at chance level. On a mixed workload most of a router's gain comes from the dataset a question belongs to, and no router meets our decision rule. IRCoT beats the hull on 2Wiki for two of three readers and in one MuSiQue cell, but chains that run its reasoning for a fixed number of steps match its accuracy at 45% to 76% of its latency, so its stopping rule adds nothing; with them in the hull it wins none of 12 tests.

[1]Code and answers: https://github.com/yasirsaleem-lab/FunnelPlan.

II. Related Work

Adaptive and multi-step retrieval. One line of work decides when to retrieve. Mallen et al. found that retrieval helps most on less popular facts, and that retrieving only when needed raises accuracy and cuts cost [2]. SKR asks the model whether it knows the answer [3], SeaKR reads uncertainty from internal states [23], and Self-RAG trains the reader to emit retrieval decisions [24]. Self-RAG needs a fine-tuned reader, so it falls outside our frozen reader setting. Verbalized confidence tends to be overconfident [25], which limits gates built on it. A second line decides how much to retrieve, from the score distribution [6] or from reader feedback [7]. A third picks a strategy, from a complexity classifier [5] or from cheap question features [26]. Iterative methods retrieve again during generation [4], [22], [27]. More context is not always better: readers miss facts in the middle of long inputs [28], accuracy can fall as passages are added [29], and models often guess when the context is not sufficient [30]. A reported benefit of random passages [31] did not hold up under changed inference settings [32].

Routing and cascades. LLM routers and cascades send easy queries to cheap models and hard ones to strong models [12], [13], [33], [34]. FrugalGPT measures its gains against single models [33]. AutoMix measures the benefit per unit cost against the line between a small and a large model [34]. Hybrid LLM and RouteLLM compare their routers with a random router at the same cost, and RouteLLM reports the share of the gap between the weak and the strong model that a router recovers [12], [13]. Our hull applies this idea to retrieval plans: it picks two plans from many rather than two models, uses measured costs, and is chosen on training folds. Systems work optimizes RAG serving cost [35], [36] and adapts the configuration per query [37]. In data management, SPARQ routes table questions by predicted cost [9] and CARROT plans retrieval under a token budget [8]. Per question plan choice is an instance of algorithm selection [38], where an oracle built from one run per instance is known to be biased upward [39]. Cost indicators can disagree with each other [40], so we report several.

Benchmarks and evaluation practice. BEIR compares retrievers zero-shot [41]. FlashRAG reimplements many RAG methods under shared settings [19]. RAGPerf profiles the latency and quality of each RAG stage [20], XRAG benchmarks RAG components [21], and BrowseComp-Plus fixes the corpus so that retrievers and search agents can be judged apart [42]. The closest study to ours compares 35 adaptive retrieval methods on six datasets with one reader and counts reader and retriever calls [10]. Experiment and benchmark papers in data management have tested learned components against strong

simple baselines, in cardinality estimation [43]–[45], query optimization [17], [18], [46] and spatial indexing [47]. Our statistics follow work on seed variance [48], test choice [49], [50] and power [51]. None of these studies compares adaptive retrieval policies with the hull of fixed plans at equal measured cost on held-out questions.

III. Problem Definition

Plans, policies and controllers. A retrieval plan p is a fixed execution template. A single-step plan is a retriever and a depth k, the number of passages given to the reader; the closed-book plan retrieves nothing ($k = 0$). A fixed two-hop plan retrieves twice with a fixed k per hop. Its second query depends on the question, but its calls are fixed in advance (Section IV). An adaptive policy π picks one plan per question, $\pi(q)$, before running it, using signals it computes first. A sequential controller such as IRCoT [22] instead decides after each step whether and how to go on. A reader r is the language model that writes the answer. For a question q, plan p and reader r we observe a success label $s(q, p, r) \in \{0, 1\}$ and a measured cost $c(q, p, r)$, the retrieval time plus the reader time in seconds. Below, the reader is fixed and we drop r.

Success and cost. On a question set Q a plan's success rate is $S(p) = \frac{1}{|Q|} \sum_{q \in Q} s(q, p)$. Its cost $C(p)$ is the mean of $c(q, p)$ over all questions of the reader and dataset. A policy's success $S(\pi)$ is the mean of $s(q, \pi(q))$. Its cost $C(\pi)$ is the mean of $C(\pi(q))$ plus its own overhead, such as a retrieval probe or a closed-book call used as a signal. The policy and the hull use the same plan costs. These include the held-out questions, but they carry cost only, never outcomes.

The fixed plan hull. Take two fixed plans a and b with $C(a) < C(b)$. A system that flips a biased coin per question, running a with probability λ and b otherwise, has expected cost $\lambda C(a)+(1-\lambda)C(b)$ and expected success $\lambda S(a)+(1-\lambda)S(b)$. It uses no information about the question. Provost and Fawcett gave the same construction for classifiers: random mixing reaches any point on the segment between two operating points [11]. The fixed plan hull is the best success that a random mix of the fixed plans $\mathcal{P}$ reaches within a budget c:

$$H(c) = \max_{w \in \Delta(\mathcal{P})} \sum_{p \in \mathcal{P}} w_p \, S(p) \quad \text{subject to} \quad \sum_{p \in \mathcal{P}} w_p \, C(p) \le c, \tag{1}$$

where $\Delta(\mathcal{P})$ is the set of distributions over $\mathcal{P}$. With one cost constraint an optimum needs at most two plans: the vertices a and b of the upper concave envelope of the points $(C(p), S(p))$ on either side of c. Between the cheapest plan and the most successful plan the budget is used in full, so the hull and the policy then have the same cost. Above the most successful plan the hull is flat and may spend less than c; this case matters for IRCoT, which costs more than every plan. No policy costs less than the cheapest plan, since it runs a plan for every question. A policy that cannot beat $H(C(\pi))$ offers nothing that a coin flip between two fixed plans would not offer within the same budget.

Gain. We score a policy by its gain over the hull at its own mean cost. Let a, b and λ be the vertices and the weight that solve (1) at $c = C(\pi)$. The gain of the policy is

$$G(\pi) = S(\pi) - [\lambda \, S(a) + (1 - \lambda) \, S(b)] \,, \tag{2}$$

the policy's success minus the hull's expected success, both measured on the same questions. Section IV-E gives the tests.

Held-out selection. Both sides choose their settings on the same data. The hull vertices, every policy setting and every threshold are chosen on training folds, separately for each reader and dataset. So the hull knows which dataset a question comes from, but nothing about the question itself; Section VI-F removes that knowledge. Gains are measured on the held-out fold, with the hull read at the policy's cost on that fold (Fig. 3). A hull chosen on the test fold would be optimistic: the best of 31 plans on a few hundred questions is inflated by selection [52]. The same bias inflates an oracle that picks the best plan per question from single noisy labels [39], so we report oracle headroom on held-out questions only (Section VII).

Hypotheses. H1: no adaptive single-step policy beats the held-out fixed plan hull at equal mean cost. H2: IRCoT, which interleaves retrieval and reasoning, beats the held-out single-step hull at its own mean cost.

IV. The FunnelPlan Benchmark

Fig. 1 shows the benchmark. Its core is a plan matrix: every question answered by every plan and every reader, with a graded answer and a measured cost in each cell. Any policy that picks a plan per question can then be replayed on the matrix without new model calls, and compared with the hull on the same questions.

A. Data

We use four open-domain question answering (QA) datasets and draw 500 questions from each with a fixed seed (42), among the questions that have a gold answer. For NQ these are the 3,452 questions of the BEIR test set, which come from the NQ dev set; 2,255 of them match an NQ-open answer by exact question text. For HotpotQA they are the 7,405 dev questions of the distractor setting, which form the BEIR test set; for MuSiQue the 2,417 answerable dev questions; for 2Wiki the 12,576 dev questions. Natural Questions (NQ) [53] is single hop; we use the open answers of [54]. HotpotQA [55] mixes one and two-hop questions. 2WikiMultihopQA (2Wiki) [56] and MuSiQue [57] are multi-hop. For NQ and HotpotQA we index the full BEIR Wikipedia corpora [41], with 2.68M and 5.23M passages. 2Wiki and MuSiQue come with paragraphs per question. We pool the unique paragraphs of the train and dev splits into one corpus per dataset, which gives 413,024 paragraphs for 2Wiki and 101,958 for MuSiQue. IRCoT also pools such paragraphs, but its corpora hold 430,225 and 139,416 paragraphs [22]. Our two corpora are therefore smaller, and our accuracies on them are not directly comparable with IRCoT's. Every plan and policy searches the same corpus, so this does not bias the comparison

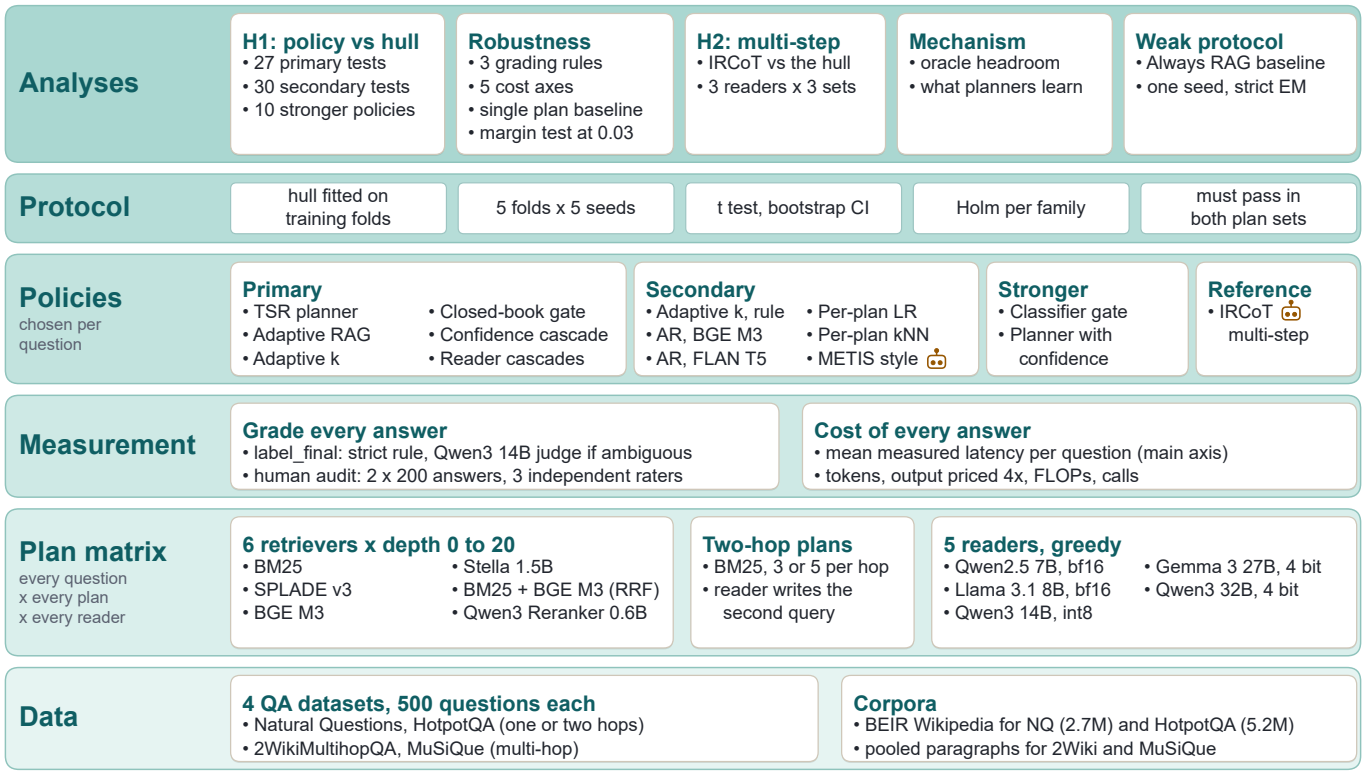


Fig. 1. The FunnelPlan benchmark, read from the bottom. Every question is answered by every plan and every reader. Each answer is graded and its cost is measured. Policies are then replayed on these answers and compared with the fixed plan hull under one held-out protocol.

with the hull. Each passage is indexed with its title. Table II summarizes the data.

TABLE II: Datasets and corpora. Each dataset contributes 500 questions drawn with a fixed seed.

Dataset	Questions	Passages	Corpus
NQ [53], [54]	single hop	2.68M	BEIR Wikipedia [41]
HotpotQA [55]	one or two hops	5.23M	BEIR Wikipedia [41]
2Wiki [56]	multi-hop	413,024	pooled paragraphs
MuSiQue [57]	multi-hop	101,958	pooled paragraphs

B. Plan Matrix

Retrievers. Six retrievers cover lexical, learned sparse, dense, hybrid and reranked search: BM25 [58]; SPLADE v3, a learned sparse model of the SPLADE family [59], [60]; BGE M3 in dense mode [61]; Stella 1.5B, a dense encoder; BGE M3 and BM25 fused with reciprocal rank fusion (RRF, $k = 60$) [62]; and Qwen3 Reranker 0.6B, which reranks the top 100 of the RRF list. The reranker's candidate pool is fixed, so depth controls only how many reranked passages reach the reader.

Depths and plans. Each retriever runs at depths $k \in \{1, 3, 5, 10, 20\}$. With the closed-book plan this gives 31 single-step plans. Qwen3 14B uses depths up to 10, since depth 20 prompts did not fit in 20 GB of memory. The two 4-bit readers use the same 25 plans as Qwen3 14B, so that the three larger readers share one plan list. We add two fixed two-hop plans, BM25 with $k = 3$ and $k = 5$ per hop. In hop 1 the reader sees the top k passages and writes one search query for the missing fact. Hop 2 retrieves k new passages for that query, and the reader answers from all $2k$. Their calls are fixed in advance, so they are plans; IRCoT, which decides after each step, is not. We report every H1 result twice: with single-step plans only, and with the two-hop plans added to the hull and to every policy.

Readers. Table III lists the five readers. Qwen2.5 7B and Llama 3.1 8B run in bf16. Qwen3 14B runs in int8 with LLM.int8() [63]. Gemma 3 27B and Qwen3 32B fit a 20 GB card only at about 4 bits, so they run as GGUF IQ4_XS files in llama.cpp. A control run measured the cost of that choice: Qwen3 14B in GGUF agreed with Qwen3 14B in int8 on 96.0% of the 12,500 HotpotQA labels. All readers decode greedily, write at most 100 new tokens and answer one question at a time. The output limit follows [32], which found that a 15 token cap changed a reported RAG effect and warned that strong quantization can do the same. Qwen3 runs with thinking disabled.

C. Measurement

Grading. Every answer is normalised as in SQuAD [64]. The strict rule counts an answer as correct if it equals a gold answer or contains one as a word sequence. String rules miss paraphrases and give credit to hedged answers [65]. We therefore define an ambiguous band: no exact or contained match but token F1 of at least 0.2, or an answer of at least

TABLE III: Models in the plan matrix. Readers run one question at a time on one RTX 3080 with its memory raised to 20 GB.

Role	Model (Hugging Face identifier)	Precision
Reader	Qwen/Qwen2.5-7B-Instruct	bf16
Reader	meta-llama/Llama-3.1-8B-Instruct	bf16
Reader	Qwen/Qwen3-14B	int8
Reader	google/gemma-3-27b-it	GGUF IQ4_XS
Reader	Qwen/Qwen3-32B	GGUF IQ4_XS
Retriever	naver/splade-v3	fp16
Retriever	BAAI/bge-m3	fp16
Retriever	NovaSearch/stella_en_1.5B_v5	fp16
Reranker	Qwen/Qwen3-Reranker-0.6B	fp16
Judge	Qwen/Qwen3-14B	int8

8 words that contains the gold. Inside the band, a local judge (Qwen3 14B) reads the question, the gold answers and the answer, and we accept when its probability of "yes" is at least 0.5. Outside the band the strict rule decides. We call the result `label_final`. Two looser string rules serve as checks: F1 $\geq$ 0.7 (strict, or token F1 of at least 0.7) and lenient (strict, or token F1 of at least 0.4). LLM judges have known biases [66], and their agreement with humans varies by task [67]. Following ARES [68], we check the judge against humans. Two raters labelled two stratified audits of 200 answers each on blank sheets, without AI help and without seeing other labels. A third rater labelled every item on which the two disagreed, and the human reference is the majority label. We fixed the band and the judge threshold after the first author had labelled the audit 1 sample himself and before the raters labelled audits 1 and 2; on the 160 ambiguous items of audit 1, thresholds from 0.3 to 0.8 give accuracies within 0.013 of each other. We report Cohen's kappa [69] and Gwet's AC1 [70], which is stable when most items fall in one class.

Cost. The main cost is the mean measured time per question: retrieval latency plus reader compute time. Retrieval is timed once per question at depth 100, after one untimed warm up call and with GPU synchronisation, and all depths of a retriever share that time. For transformers readers we time the generate call; for llama.cpp readers we use the server's own prompt and generation time. Retrieval and reading ran in separate passes, each with the GPU to itself, and every cell was timed once. Section VIII compares this replay with a live end-to-end run. Latency depends on hardware, and efficiency claims can change with the cost metric [40]. We therefore repeat the primary tests under four costs that do not depend on the machine. Tokens is prompt plus generated tokens. Tokens $\times 4$ prices each generated token at four input tokens. FLOPs is $2\times$ reader parameters $\times$ tokens, plus the reranker and the dense query encoders (dense search over the corpus is not counted). Calls counts reader calls plus retrieval, fusion and rerank calls.

D. Policies

Fig. 2 shows how each policy type chooses a plan. Every policy chooses its settings and thresholds on the training folds, and learned models train on the 400 training questions of one reader and dataset. Each tuned setting maximises the policy's gain over the training fold hull at the policy's own cost; classifiers fit their usual loss. We call a method reproduced when we run its published rule on our plans, adapted when we change a component, and inspired when we build it from a published idea; each description below says which.

Primary policies (five, plus two reader cascades). The *TSR planner*, our own planner, predicts the task success rate (TSR) of every plan from cheap decision-time features, namely question text features and a BM25 probe whose cost is charged. It uses a plan prior plus gradient-boosted trees (XGBoost [71]) and runs the plan that maximises predicted success minus μ times cost. *Adaptive RAG* [5] routes each question to no retrieval, a single-step plan or the strongest plan. Its classifier is trained on silver labels: the simplest rung that answers correctly, and the strongest rung when none does. We replace the paper's T5 classifier with TF IDF features and logistic regression (adapted); the secondary family adds a T5 classifier back. *Adaptive k* [6] sets the depth per question at the largest drop in one retriever's sorted similarity scores; in this family we search the top 20 and map the cut to the nearest labelled depth (adapted). The *closed-book gate* runs the reader without passages and retrieves only when the answer's mean token log probability is below a threshold. It follows the self-knowledge idea of SKR [3] and the adaptive retrieval motivation of [2], with model confidence [25] in place of entity popularity, which our datasets lack (inspired). The *confidence cascade* runs a cheap plan and escalates to a strong plan when the answer's confidence is low, in the spirit of FLARE's confidence trigger [4] (inspired). Two *reader cascades* pass the least confident answers of Qwen2.5 7B to Llama 3.1 8B or to Qwen3 14B [33], [34] (inspired); their hull covers the fixed plans of both readers. This family has 27 tests: 5 policies $\times$ 5 readers, plus 2 cascades.

Secondary policies (30 tests). To test whether the null result comes from weak routers, we add stronger variants: Adaptive k with the published rule (largest drop within the top 90% of 100 candidates from BGE M3 or Stella, the retriever chosen on the training folds, plus 5 passages, mapped to the nearest labelled depth with ties going deeper; reproduced); Adaptive RAG with logistic regression on BGE M3 embeddings and with a fine-tuned FLAN-T5 Large classifier [72]; per-plan success models with logistic regression and with k-nearest neighbours on question embeddings [26] (adapted; the replay does not charge the query embedding, which Section VI-E measures); and a METIS style router (inspired). In METIS [37], a larger model profiles each question (joint reasoning, complexity, pieces of information) and rules prune the configuration space. Our router uses Qwen3 32B as the profiler, maps the profile to our plans, and charges no cost for the profiler, which favours it. It falls back to the median profile of the training folds when the profile has low confidence. It borrows the METIS profile but not its serving system, so it does not test METIS itself.

Stronger policies (10 tests). Recent work argues that adaptive retrieval needs better uncertainty signals [10]. We add a classifier gate, gradient-boosted trees on five closed-

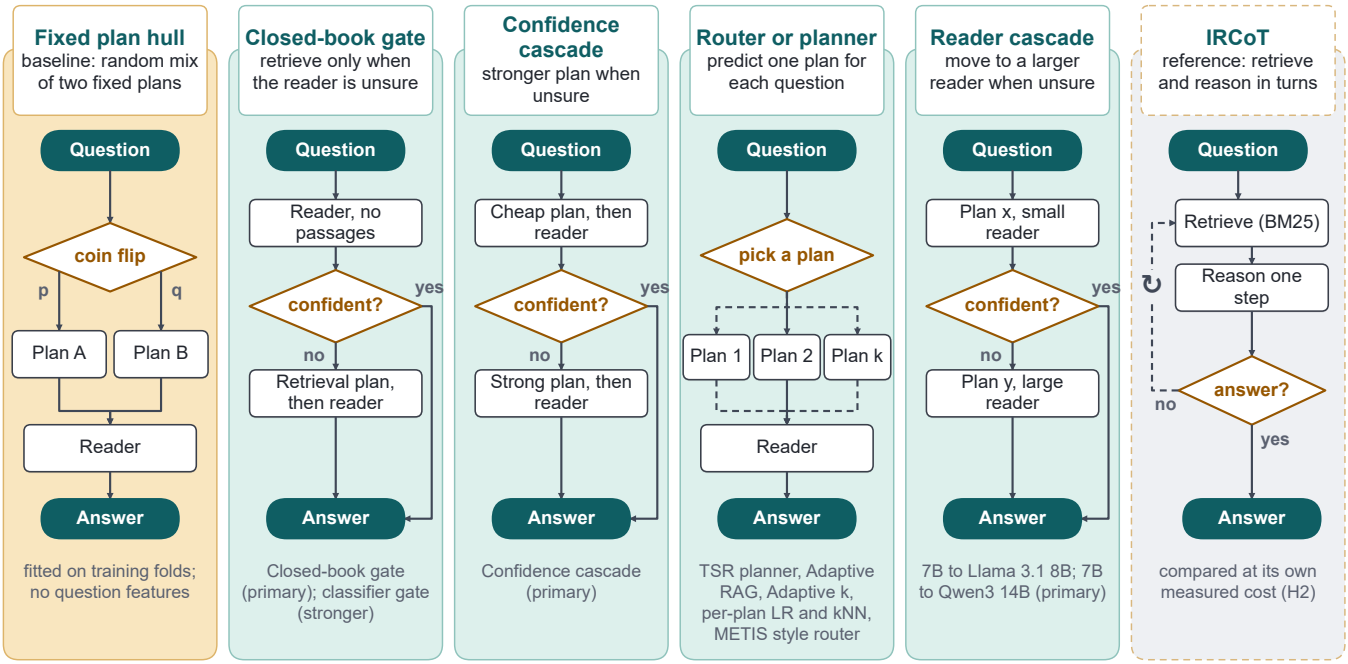


Fig. 2. The baseline and the policy families as flow charts. Teal boxes start and end each flow, amber diamonds are the decisions a policy makes, and dashed arrows are paths not taken. The hull mixes two fixed plans at random and uses no question features. The footer of each column lists the policies of that type.

book confidence features with a threshold set on cross-fitted predictions, and the TSR planner with the same features added.

Multi-step reference. IRCoT [22] alternates one reasoning sentence with one BM25 retrieval, using the first generated sentence as the next query. It stops when the reasoning contains "answer is" or after 8 steps, and collects at most 15 paragraphs. We use the authors' demonstrations, $K = 4$ paragraphs per step (from the paper's grid $\{2, 4, 6, 8\}$), and the same answer prompt as the plans (reproduced; unlike the paper we do not cap the prompt at 6,000 tokens). It is a sequential controller, not a plan; we compare it with the single-step hull at its own mean cost (H2). We run IRCoT for the three readers served by Transformers; our plan listed the two 4-bit readers as optional, and we did not run them. On NQ, IRCoT uses the HotpotQA demonstrations, as Adaptive RAG does [5]; its NQ prompt file is the same file.

Adaptive RAG as published. This variant follows [5] as closely as our plans allow. Its ladder is no retrieval, BM25 with 5 passages and IRCoT. The silver label of a question is the simplest rung that answers it; a question that no rung answers gets the rung of its dataset type, one retrieval for NQ and IRCoT for the three multi-hop datasets. One T5 Large classifier per reader learns these labels from the question text over the 2,000 pooled questions, with AdamW, a learning rate of 3×10^{-5}, batches of 32 and inputs of up to 384 tokens. The number of epochs (15 to 25) is chosen per reader from $\{15, 20, \ldots, 35\}$ on an inner split of the first training fold. Its measured inference time, 122 ms per question, is charged. On held-out folds it predicts the silver label for 61% to 67% of questions.

Fixed step chains. To separate IRCoT's stopping rule from its extra steps, a fixed chain F_T runs IRCoT's loop for exactly T retrievals, $T = 1$ to 4, with the stopping rule turned off, and then answers with the same prompt (Section VI-G).

E. Protocol

Input: labels $s(q, p)$ and costs $c(q, p)$ of one reader on one dataset (500 questions); one policy.
1. $C(p) \leftarrow$ mean of $c(q, p)$ over the 500 questions, for every plan p.
2. For each seed $t = 1, \ldots, 5$, split the questions into five folds with seed t.
3. For each fold F, with training set T (the other four folds):
4. Fit the policy's models, settings and thresholds on T only.
5. Hull vertices $\leftarrow$ upper concave envelope of $(C(p), S_T(p))$.
6. Run the policy on F; $c_F \leftarrow$ mean over $q \in F$ of $C(\pi(q))$ plus overhead.
7. Vertices a, b on either side of c_F and λ with $\lambda C(a) + (1 - \lambda)C(b) = c_F$;
above the last vertex, $a = b =$ the most successful plan.
8. $g_t(q) \leftarrow s(q, \pi(q)) - [\lambda S_F(a) + (1 - \lambda) S_F(b)]$ for $q \in F$ (S_F: mean on F).
9. $g(q) \leftarrow$ mean of $g_t(q)$ over the seeds. Pool four datasets: 2,000 gains.
10. Per-question test on the 2,000 gains; corrected test on the 25 fold gains per dataset; Holm.

Fig. 3. The held-out protocol for one policy, reader and dataset.

Fig. 3 gives the protocol. Each policy is fitted on four folds and scored on the fifth. We repeat this with five seeds, each changing the fold split and the model seed, and average each question's gain over seeds. Pooling four datasets gives 2,000 gains per test.

Tests. We use two tests. The per-question test is a one-sample t test on the 2,000 gains [50] with a 95% bootstrap interval [73]. A question's gain is the policy's success on it minus the hull's mean success on its fold (Fig. 3), and the

fitted policies and hulls are treated as fixed. We do not pair the arms by question: pairing halves the standard error, to only 0.40 to 0.45 of the refit spread on the four tests near a decision below. The corrected test counts the variance from refitting. It takes the 25 fold gains of each reader and dataset, scales their variance by $1/25 + n_{\text{test}}/n_{\text{train}}$ as in the corrected resampled t-test [74], [75], and averages the four datasets. A fold gain compares both arms on the same questions, so this test is paired by fold, and its intervals can be narrower than the per-question ones. The corrected test treats the 25 fold gains as exchangeable and corrects their correlation only through the training overlap $n_{\text{test}}/n_{\text{train}}$, so it is approximate [74]; this is why we check it against a refit bootstrap.

Both tests may still be too narrow. A refit bootstrap draws 100 resamples of the questions, keeps the copies of a question in one fold, and repeats steps 3 to 8. With one seed per resample, on four tests close to a decision and 12 drawn at random, its spread is 0.96 to 1.47 times the per-question standard error and 1.03 to 2.61 times the corrected one. One seed adds split noise that our average over five seeds lacks, so we also run the corrected test with every standard error inflated by 2.61, and we rerun the bootstrap with five seeds per resample on the 12 primary tests whose inflated bound exceeds 0.03 (Section VI-C). Holm's correction runs within each family [76]. One rule decides every verdict: a policy beats the hull, or loses to it, only if in both plan sets its Holm-adjusted p is below 0.05 and its interval lies on the same side of zero under all three tests; any other count names its test. This rule detects a pooled gain of about 0.048 with 80% power, and about 0.068 at Holm's strictest step. IRCoT fits nothing but the hull, so for IRCoT a refit bootstrap of each test replaces the two corrected tests. Tables and figures show the per-question test unless they say otherwise.

A null result is not evidence of no effect [77]. We therefore test non-superiority at a margin of 0.03, a margin we chose after seeing the main results. It matches the gains that adaptive methods report over fixed retrieval: 0.019 to 0.037 for Adaptive RAG over single-step retrieval [5], and 0.016 to 0.082 for the best adaptive method over Always RAG with Llama 3.1 8B [10]. It does not exclude the smaller gains of plain uncertainty methods (0.004 to 0.016 in [10]). The extended version lists every bound, so other margins can be applied. We also report the minimum detectable effect at 80% power [51]: 0.025 to 0.029 for a single pooled primary test, and 0.037 to 0.044 at the strictest Holm step.

V. Experimental Setup

Hardware and software. All answers were generated on one desktop with an NVIDIA GeForce RTX 3080 whose memory was raised to 20 GB (driver 617.14), an AMD Ryzen 7 5700X (8 cores) and 64 GB of RAM. Readers answer one question at a time with no batching, so every latency is a per-question latency. The bf16 and int8 readers run in PyTorch 2.7.1 (CUDA 11.8 build) and Transformers 4.51.0, and the 4-bit readers in llama.cpp (build b11491). BM25 uses bm25s 0.3.11. Dense search is exact: BGE M3 and Stella store 1,024 dimensional fp16 vectors, and the query is scored against every passage on the GPU, or in chunks on the CPU when the vectors do not fit. The learned policies use XGBoost 3.2.0 [71] and scikit-learn 1.7.2.

Scale. The plan matrix holds 274,000 single-step answers: 62,000 for each of the two small readers and 50,000 for each of the other three. It also holds 20,000 two-hop answers, 6,000 IRCoT runs and 24,000 fixed chain answers (three readers on four datasets). Each cell stores the answer text, its grades, its latency, and its token and call counts. Generation took several GPU days per large reader. Every analysis in this paper reruns from the stored answers on a CPU; the core analyses take about 7 hours.

Prompts. The repository holds every prompt. The reader prompts, with and without passages, are in `tsr/agent_runner.py`. The judge prompt, which the outside judge also uses, is in `scripts/07_grading_full.py`, and the two-hop query prompt is in `scripts/05_two_hop_plans.py`. IRCoT uses the authors' demonstration files in `data/ircot_prompts/`. The chain-of-thought closed-book prompt behind Table VI uses the Adaptive RAG demonstration files in `data/external/adarague/`, run by `scripts/31_closed_book_prompt.py`.

Implementation checks. Two checks test whether our policies and readers behave as published. First, under the weak protocol of Section VI-B, Adaptive RAG beats Always RAG by $+0.052$ on average and in 17 of 20 reader and dataset cells. This matches the direction reported by its authors [5] and by Moskvoretskii et al., who find Adaptive RAG at or above Always RAG on all four of our datasets [10]. This shows that our version reproduces the published direction of the gain over Always RAG, not that it keeps the original router's quality. Second, we compare Llama 3.1 8B with Table 1 of [10] under their answer contained accuracy (Table IV). With BM25 at depth 10 our accuracy is within 0.03 of their Always RAG value on every dataset (for example, 0.480 against 0.496 on NQ). Our closed-book accuracy is 0.06 to 0.10 lower than their Never RAG value. The corpus cannot cause this gap, so we reran their closed-book setup: the few-shot chain-of-thought prompt of Adaptive RAG, on their 500 test questions per dataset and on ours. Their stored answers give their Table 1 values on HotpotQA, 2Wiki and MuSiQue, but 0.400 rather than 0.446 on NQ. On the same questions their prompt beats ours by 0.05 to 0.12, while the question sample moves our accuracy by at most 0.056, so the prompt explains most of the gap. Their prompt also takes 2.77 s per answer against our 0.46 s. We replayed the primary policies, the classifier gate and the planner with self-knowledge features for Llama 3.1 8B with these stronger closed-book answers. No policy turns positive and every upper bound stays below 0.016, even when the new answers keep our old cost.

Table V lists the same comparison as written by the analysis script, with both BM25 depths next to the published values. Each row is one dataset. The first two value columns compare the published Never RAG accuracy with our closed-book plan.

TABLE IV: Llama 3.1 8B against the values published by Moskvoretskii et al. (their Table 1), answer contained accuracy. Pub.: published. Their corpus and prompt differ from ours.

	Never RAG		Always RAG (BM25)		
Dataset	Pub.	Ours	Pub.	Ours k=5	Ours k=10
NQ	0.446	0.350	0.496	0.426	0.480
HotpotQA	0.286	0.210	0.410	0.382	0.398
2Wiki	0.318	0.250	0.374	0.332	0.346
MuSiQue	0.106	0.046	0.100	0.116	0.128

The last three compare the published Always RAG accuracy with our BM25 plans at depths 5 and 10. The published retrieval depth is not stated, so we show both depths. On NQ, HotpotQA and 2Wiki, depth 10 is closer to the published value than depth 5. On MuSiQue both of our BM25 plans are slightly above it (0.116 and 0.128 against 0.100).

Table VI gives the closed-book check behind the comparison above. Our run of their prompt on their questions agrees in correctness with their stored answers on 92.5% of questions; the chain-of-thought text often differs. On our questions and under `label_final`, their prompt reaches 0.518, 0.370, 0.344 and 0.148 on NQ, HotpotQA, 2Wiki and MuSiQue, against 0.418, 0.280, 0.154 and 0.062 with ours. Our prompt abstains on 0.6% to 9.8% of questions, and theirs on 0.4% to 2.8%. We cannot explain the NQ gap between their stored answers and their Table 1 from their release. Table XXVIII gives the replay with these answers.

VI. Results

Table VII gives the verdict of every test family. Each row counts the Holm survivors whose interval lies above zero (Pos) or below zero (Neg) in each plan set, under the per-question test unless the row says otherwise. By the decision rule of Section IV, a policy beats the hull only with a positive survivor in both plan sets. No row against the hull has one.

A. The Fixed Plans

Table VIII shows the plans that every policy chooses from, at depth 5 and averaged over the five readers. No retriever wins on every dataset. Stella is best on NQ (0.734), and the reranker is best among single-step plans on the three multi-hop datasets. Retrieval time differs by two orders of magnitude, from 0.017 s for BM25 to 1.378 s for the reranker. Closed-book answers reach 0.080 to 0.369, so the readers know a part of the answers without retrieval.

Table IX shows the room for a policy: the share of questions that at least one single-step plan answers exceeds the best single plan by 0.14 to 0.29. This in-sample oracle is optimistic [39]; Section VII measures it on held-out questions. The two-hop plans win only on 2Wiki (and by 0.004 for Gemma 3 27B on MuSiQue).

Table X repeats this oracle over the same 25 plans for every reader (depth up to 10), so that readers can be compared. Each row is one reader and dataset. Closed-book is the success without retrieval, and Best fixed plan is the most successful single plan on all 500 questions. Oracle is the share of questions that at least one of the 25 plans answers. Headroom is Oracle minus Best fixed plan, and Unsolved is one minus Oracle. The headroom over the best fixed plan is 0.138 to 0.272. It is largest on MuSiQue for every reader (0.252 to 0.272) and smallest on NQ and HotpotQA (0.138 to 0.214). The share of questions that no plan answers ranges from 0.072 on NQ to 0.518 on MuSiQue. Fig. 4 averages these numbers over the four datasets. For every reader the oracle reaches 0.69 to 0.76, against 0.48 to 0.55 for the best fixed plan. Closed-book success is lowest on MuSiQue (0.056 to 0.106), so most MuSiQue answers need retrieval. MuSiQue also leaves the most questions that no plan answers. A policy can only gain on questions that some plan answers and the best fixed plan misses, which is the headroom column.

B. The Baseline Changes the Verdict

We first score the same answers under the protocol common in the literature. A policy wins a cell if its accuracy beats both Always RAG (BM25 at depth 5) and Never RAG (closed-book), with no cost matching, one seed and the strict rule. Under this protocol the five primary policies win 64 of 100 policy, reader and dataset cells (Fig. 5 and Table XI). Only 24 of these wins are also cheaper than Always RAG. Under our protocol the same policies win none. The confidence cascade gains $+0.083$ over Always RAG on average, yet it costs 1.84 times as much and gains $+0.002$ over the hull. The confidence cascade wins 18 of 20 cells under the weak protocol and Adaptive RAG 17 of 20, though Adaptive RAG loses 0.045 to the hull on average. Only the closed-book gate loses to Always RAG under the weak protocol, by 0.069 on average. The answers are identical in both protocols, but the protocols differ in several ways at once. Table XII changes one factor at a time. The comparator does most of the work: wins fall from 64 to 54 against the best single plan at the same cost and to 30 against the hull. This split depends on the order of the steps, so Table XV also holds the test fixed: under the pooled test the same policies beat Always RAG and Never RAG in 23 of 27 tests in each plan set, the best single plan in 5 and 7, and the hull in none. The four tests they lose against Always RAG are the closed-book gate, which loses by 0.05 to 0.15. Five seeds and the final labels change little (27 and 28). The last three rows change only the test, and each costs power: the minimum detectable effect grows from 0.055 per cell to 0.085 with Holm over 100 cells. If every cell gained 0.02, the per-cell Holm rule would find fewer than one win. So these rows show that single cells cannot settle the question, not that the 28 wins were false; the pooled test, with an effect of 0.038 detectable at Holm's strictest step, finds none of 25.

Table XIII gives the power of each decision rule in Table XII, from the per-question standard errors. A single cell detects a true gain of 0.055 with 80% power, and 0.085 after Holm's correction over the 100 cells; pooled over datasets, the values are 0.027 and 0.038. If every cell had a true gain of 0.03, Holm over the 100 cells would find about 3 wins, and pooled Holm about 13 of 25. So the per-cell rows of the

TABLE V: Llama 3.1 8B against published Never and Always RAG (answer contained accuracy).

Dataset	Published Never RAG	Ours, closed book	Published Always RAG	Ours, BM25 k=5	Ours, BM25 k=10
NQ	0.446	0.350	0.496	0.426	0.480
HotpotQA	0.286	0.210	0.410	0.382	0.398
2Wiki	0.318	0.250	0.374	0.332	0.346
MuSiQue	0.106	0.046	0.100	0.116	0.128

TABLE VI: Closed-book accuracy of Llama 3.1 8B under answer contained accuracy, the metric of Moskvoretskii et al. [10]: their prompt (the few-shot chain-of-thought prompt of Adaptive RAG) against ours, on their 500 test questions per dataset and on ours. Abstain: share of refusals; time per answer. Their stored answers give their Table 1 values on the three multi-hop datasets but not on NQ.

Run	NQ	HotpotQA	2Wiki	MuSiQue	Abstain	Time (s)
Their stored answers, their questions	0.400	0.286	0.318	0.106	0.009	
Their prompt, our run, their questions	0.414	0.286	0.342	0.104	0.013	2.66
Our prompt, their questions	0.294	0.232	0.260	0.038	0.059	0.56
Our prompt, our questions (paper)	0.350	0.210	0.250	0.046	0.053	0.46
Their prompt, our questions	0.442	0.284	0.342	0.118	0.012	2.77
Their Table 1, Never RAG	0.446	0.286	0.318	0.106		

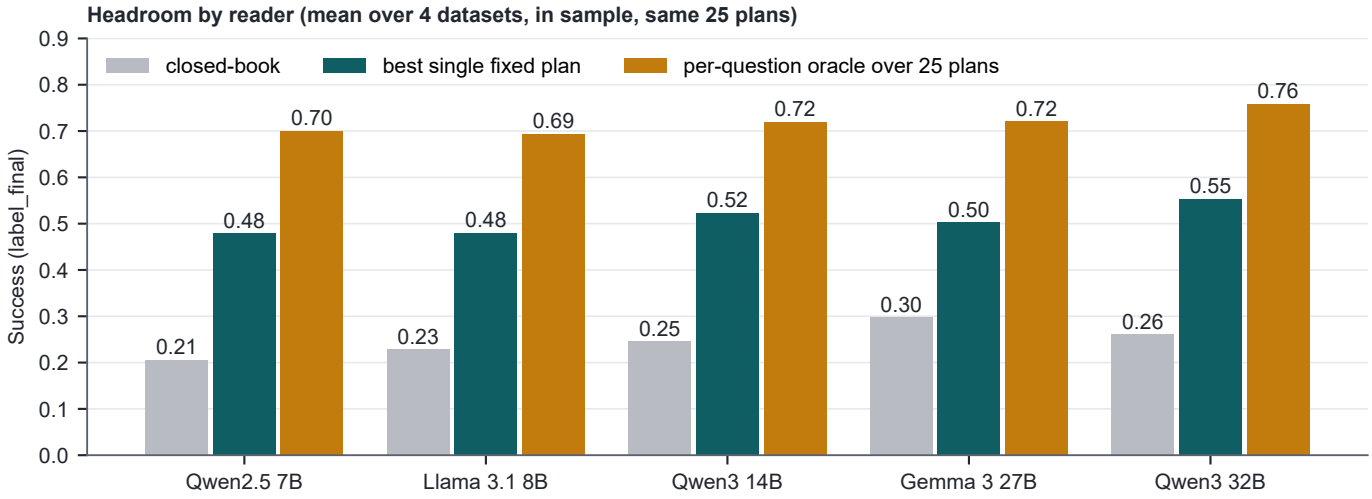


Fig. 4. In-sample headroom by reader, mean over the four datasets, over the same 25 plans for every reader: closed-book, the best single fixed plan, and a per-question oracle that picks a plan that answers.

ablation cannot detect gains of the size our margin excludes, while the pooled rows can.

Weak baselines inflated progress in information retrieval [14], [15] and in recommendation [16], and single seed comparisons are unstable [48]. Adaptive retrieval shows the same pattern.

Table XIV gives the per policy numbers behind Table XI: the margin over Always RAG and over Never RAG, the cost relative to Always RAG, and the gain under our protocol. Each row averages one policy over its 20 reader and dataset cells. Every policy beats Never RAG by a wide margin ($+0.052$ to $+0.204$), the comparison that makes adaptive retrieval look strong. Against the hull the same policies range from -0.045 to $+0.003$.

A baseline without mixing. The row marked † replaces the hull with the best single fixed plan at the same cost, the comparison most papers make. Against it, five policies win with single-step plans and seven with the two-hop plans, by up to $+0.061$ and $+0.080$. Four of the five single-step winners are the TSR planner, and the fifth is the closed-book gate on Qwen3 32B. These wins match what a coin flip between two fixed plans gains over the best single plan at the same cost: for the TSR planner on Qwen3 32B, $+0.061$ against $+0.056$. Averaged over the 27 tests, random mixing alone gains $+0.017$ (single-step) and $+0.023$ (two-hop). A policy that looks better than the best fixed plan has, in these cases, learned to mix plans, not to match plans to questions. Table XV gives these counts for both plan sets, next to the counts against Always RAG and Never RAG under the same test, with the mean value of non-adaptive mixing; the largest gains against the best single plan are $+0.061$ and $+0.080$. The first count is against the hull and the second against the best single plan at or below the policy's cost; the second is what most papers would report. The mixing column is what a coin flip between two fixed plans gains over the best single plan at the same cost, averaged over the 27 tests. It is $+0.017$ with single-step

TABLE VII: Every test family and robustness check. Pos and Neg count Holm survivors whose 95% interval lies above or below zero; Max is the largest pooled gain. Rows marked ‡ use the corrected test, the others the per-question test. Every row compares with the fixed plan hull at the policy's mean cost except the row marked †, which compares with the best single fixed plan at the same cost (no mixing), and the rows marked §, which match the hull to the policy's 95th or 99th percentile latency on the training questions (reader cascades left out).

		Single-step plans		With two-hop plans	
Test family	Tests	Pos/Neg	Max	Pos/Neg	Max
Primary: latency (main)	27	0/4	+0.013	0/4	+0.017
Primary: tokens	27	0/5	+0.012	0/5	+0.013
Primary: tokens, output x4	27	0/5	+0.012	0/5	+0.014
Primary: FLOPs	27	0/5	+0.016	0/5	+0.011
Primary: calls	27	0/5	+0.006	0/8	+0.008
Primary: P95 latency hull§	25	0/8	+0.019	0/7	+0.017
Primary: P99 latency hull§	25	0/9	+0.020	0/9	+0.018
Primary: best single plan†	27	5/4	+0.061	7/5	+0.080
Primary: strict grading	27	0/5	+0.010	0/5	+0.009
Primary: lenient grading	27	0/4	+0.019	0/5	+0.022
Secondary family	30	0/6	+0.029	1/5	+0.030
Stronger policies	10	0/0	+0.001	0/0	−0.002
Primary: corrected test‡	27	0/5	+0.013	0/5	+0.017
Primary: corrected, x2.61‡	27	0/0	+0.013	0/0	+0.017
Primary: outside judge‡	27	0/6	+0.015	1/6	+0.020
Secondary, charged‡	30	0/6	+0.011	0/4	+0.014

Primary upper bounds below 0.03: per question 27 and 26 of 27; corrected 27 and 26; corrected, errors x2.61: 22 and 20 (single-step and with two-hop); the 12 tests above 0.03 there refitted with five seeds per resample: 27 and 26 (largest refit bound 0.032).

TABLE VIII: Fixed plans at depth 5: success averaged over the five readers (in sample, `label_final`), and mean retrieval time per question. Two-hop: BM25 with 5 passages per hop.

Plan	NQ	HotpotQA	2Wiki	MuSiQue	Retr. (s)
Closed-book	0.369	0.280	0.261	0.080	0.000
BM25	0.490	0.453	0.372	0.172	0.017
SPLADE v3	0.688	0.487	0.360	0.225	0.610
BGE M3 (dense)	0.696	0.504	0.380	0.226	0.035
Stella 1.5B	0.734	0.524	0.361	0.268	0.057
BGE M3 + BM25 (RRF)	0.640	0.512	0.390	0.218	0.059
Qwen3 Reranker	0.703	0.577	0.402	0.275	1.378
BM25, two-hop	0.574	0.547	0.497	0.258	0.029

plans and +0.023 with the two-hop plans, so a policy can beat the best single plan without using any question information.

Scoring on the tuning data. On the training folds, where the policy and the hull were fitted, Adaptive RAG gains +0.072 and the TSR planner +0.029. On held-out questions these become −0.045 and +0.003. An evaluation that tunes and tests on the same questions would report both as wins [52].

Adaptive RAG as published. Adaptive RAG [5] trains a T5 Large classifier on silver labels from a mix of datasets and sends each question to no retrieval, BM25 with five passages, or IRCoT. We ran it this way for the three IRCoT readers on the pooled 2,000 questions, with its published settings and its classifier's 122 ms per question charged (Section IV). It sends 53% to 59% of questions to IRCoT. Against Always RAG, the comparison its paper makes, it gains +0.070 to +0.091, positive for all three readers under the per-question and corrected tests, at 23 to 37 times the latency. Against the global hull at its own cost (Section VI-F) it scores 0.023 to 0.061 lower, a gap that the per-question test finds in all 6 tests and the corrected test in 4; against the label hull and a hull that may also run IRCoT, the corrected test finds it in all 6. At Adaptive RAG's cost the global hull is one plan in every fold, the reranker at depth 10, which takes 2.0 to 3.5 s, so this gap compares Adaptive RAG with a stronger retriever, not with a mix. A random mix of Adaptive RAG's own rungs, BM25 at depth 5 and IRCoT, at its cost isolates the classifier: Adaptive RAG gains +0.002 to +0.016 over it, and no test finds a difference (largest corrected interval $[-0.000, +0.032]$). Its classifier adds nothing over a coin flip between its rungs; its win over Always RAG comes from running IRCoT on more than half of the questions. Under the corrected test with errors inflated by 2.61, no comparison of Adaptive RAG with a hull is significant in either direction. Table XVI gives the gains per reader. Its gains over Never RAG, +0.184 to +0.236, pass all three tests in both plan sets, so the published comparison would credit it with a large win.

C. No Primary Policy Beats the Fixed Plan Hull

Fig. 6 shows what a policy must beat. Each grey point is one fixed plan, and the line joins the plans that a coin flip can mix. Most policies land near the line, and some land well below it. Adaptive RAG, for example, pays for strong plans on many questions but does not convert that cost into extra correct answers.

Fig. 6 shows what a policy must beat. Each grey point is one fixed plan, and the line joins the plans that a coin flip can mix. Most policies land near the line, and some land well below it. Adaptive RAG, for example, pays for strong plans on many questions but does not convert that cost into extra correct answers.

Fig. 7 shows all 27 primary tests in both plan sets. No test is positive after Holm's correction. The largest gains are +0.013 with single-step plans (confidence cascade, Qwen2.5 7B) and +0.017 with the two-hop plans (reader cascade from

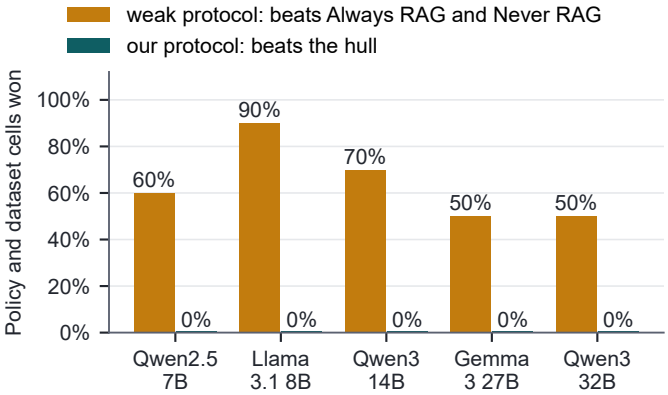


Fig. 5. Share of policy and dataset cells won by the same five primary policies under two protocols. Weak protocol: beat both Always RAG (BM25, $k = 5$) and Never RAG in accuracy, with no cost matching, one seed and strict grading. Our protocol: beat the held-out hull at equal mean latency.

TABLE IX: Fixed plans per reader and dataset (in sample, `label_final`). CB: closed-book. Best: best single-step plan. Orc.: share of questions that at least one single-step plan answers. 2h: best fixed two-hop plan minus the best single-step plan.

	NQ				HotpotQA				2Wiki				MuSiQue			
Reader	CB	Best	Orc.	2h	CB	Best	Orc.	2h	CB	Best	Orc.	2h	CB	Best	Orc.	2h
Qwen2.5 7B	0.284	0.730	0.922	−0.166	0.226	0.530	0.758	−0.038	0.258	0.406	0.640	+0.038	0.056	0.262	0.548	−0.016
Llama 3.1 8B	0.418	0.698	0.910	−0.146	0.280	0.588	0.778	−0.066	0.154	0.406	0.638	+0.056	0.062	0.228	0.498	−0.030
Qwen3 14B	0.334	0.776	0.914	−0.186	0.266	0.596	0.772	−0.020	0.306	0.434	0.654	+0.120	0.076	0.286	0.538	−0.028
Gemma 3 27B	0.452	0.726	0.926	−0.156	0.338	0.612	0.754	−0.052	0.292	0.388	0.644	+0.118	0.106	0.284	0.556	+0.004
Qwen3 32B	0.358	0.760	0.928	−0.166	0.288	0.608	0.790	−0.024	0.294	0.502	0.700	+0.098	0.100	0.344	0.616	−0.032

TABLE X: Headroom per reader and dataset (in sample, 25 plans).

Reader	Dataset	Closed-book	Best fixed plan	Oracle	Headroom	Unsolved
Qwen2.5 7B	2Wiki	0.258	0.392	0.614	0.222	0.386
Qwen2.5 7B	HotpotQA	0.226	0.530	0.744	0.214	0.256
Qwen2.5 7B	MuSiQue	0.056	0.262	0.530	0.268	0.470
Qwen2.5 7B	NQ	0.284	0.730	0.914	0.184	0.086
Llama 3.1 8B	2Wiki	0.154	0.406	0.626	0.220	0.374
Llama 3.1 8B	HotpotQA	0.280	0.588	0.766	0.178	0.234
Llama 3.1 8B	MuSiQue	0.062	0.228	0.482	0.254	0.518
Llama 3.1 8B	NQ	0.418	0.698	0.902	0.204	0.098
Qwen3 14B	2Wiki	0.306	0.434	0.654	0.220	0.346
Qwen3 14B	HotpotQA	0.266	0.596	0.772	0.176	0.228
Qwen3 14B	MuSiQue	0.076	0.286	0.538	0.252	0.462
Qwen3 14B	NQ	0.334	0.776	0.914	0.138	0.086
Gemma 3 27B	2Wiki	0.292	0.388	0.644	0.256	0.356
Gemma 3 27B	HotpotQA	0.338	0.612	0.754	0.142	0.246
Gemma 3 27B	MuSiQue	0.106	0.284	0.556	0.272	0.444
Gemma 3 27B	NQ	0.452	0.726	0.926	0.200	0.074
Qwen3 32B	2Wiki	0.294	0.502	0.700	0.198	0.300
Qwen3 32B	HotpotQA	0.288	0.608	0.790	0.182	0.210
Qwen3 32B	MuSiQue	0.100	0.344	0.616	0.272	0.384
Qwen3 32B	NQ	0.358	0.760	0.928	0.168	0.072

TABLE XI: The same answers under two protocols, mean over 20 reader and dataset cells. Weak protocol: accuracy margin over Always RAG (BM25, $k = 5$) and Never RAG, cost relative to Always RAG, and cells that beat both (strict grading, one seed). Ours: gain over the held-out hull at equal latency.

	Weak protocol				Ours
Policy	vs Always	vs Never	Cost	Wins	Gain
TSR planner	+0.054	+0.175	1.12×	12/20	+0.003
Adaptive RAG	+0.052	+0.173	1.75×	17/20	−0.045
Adaptive k	+0.029	+0.150	0.83×	12/20	−0.018
Closed-book gate	−0.069	+0.052	1.32×	5/20	−0.002
Confidence cascade	+0.083	+0.204	1.84×	18/20	+0.002

Qwen2.5 7B to Qwen3 14B). Under the per-question test, four tests are significantly negative in each plan set, all Adaptive RAG, with losses of up to 0.069, though none is negative under all three tests; it pays for strong plans on many questions without matching them to the questions that need them. The corrected test adds Adaptive k for Llama 3.1 8B. Adaptive k is below the hull for every reader, by 0.011 to 0.025, though no single test survives correction. The TSR planner and the closed-book gate stay within 0.007 of the hull. Table XVII lists every gain behind Fig. 7, together with the secondary and stronger families.

Table XVIII gives every primary test with its 95% interval and Holm-adjusted p value in both plan sets. Each row is one test: a policy with one reader, or a reader cascade. The gain columns give the pooled gain over the held-out hull with its 95% bootstrap interval, and the Holm columns give the adjusted p value within the family of 27 tests. A test counts as a win only if both plan sets show $p < 0.05$ with the interval above zero. Every interval of the TSR planner, the closed-book gate, the confidence cascade and the reader cascades contains zero. With single-step plans the four Adaptive RAG losses are Llama 3.1 8B (-0.059), Qwen3 14B (-0.052), Gemma 3 27B (-0.032) and Qwen3 32B (-0.060). With the two-hop plans they are Qwen2.5 7B (-0.051), Llama 3.1 8B (-0.069), Qwen3 14B (-0.058) and Qwen3 32B (-0.052). Two more Adaptive RAG tests have intervals below zero but do not survive the correction: Qwen2.5 7B with single-step plans ($p = 0.382$) and Gemma 3 27B with the two-hop plans

TABLE XII: The weak protocol turned into ours one factor at a time on the same 100 cells (5 primary policies, 5 readers, 4 datasets); the last two rows pool over datasets (25 tests).

Step	Wins
weak: beats Always RAG and Never RAG on accuracy (strict, seed 0, no cost matching)	64 of 100
comparator: best single plan at the same cost (strict, seed 0)	54 of 100
comparator: fixed plan hull at the same cost (strict, seed 0)	30 of 100
seeds: 5 fold splits instead of 1 (strict)	27 of 100
grading: label_final instead of strict	28 of 100
decision: 95% interval above 0 and $p < 0.05$ per cell	1 of 100
decision: Holm over the 100 cells	0 of 100
pooled, gain above 0	8 of 25
pooled, Holm	0 of 25

TABLE XIII: Power of the decision rules of the protocol ablation: minimum detectable effect at 80% power (median over tests), and the number of tests expected to win if every cell had a true gain of 0.02 or 0.03 (normal approximation; Holm rows use the strictest step).

Decision rule	Tests	MDE (median)	Expected wins at 0.02	Expected wins at 0.03
per cell, 95% interval above 0 and $p < 0.05$	100	0.055	18.5	35.5
per cell, Holm over 100 (strictest step, Bonferroni bound)	100	0.085	0.8	3.3
pooled over datasets, $p < 0.05$	25	0.027	13.6	21.8
pooled, Holm over 25 (strictest step, Bonferroni bound)	25	0.038	3.9	12.6

TABLE XIV: Weak protocol (strict rule, one seed, no cost matching) against ours.

Policy	Margin vs Always RAG	Margin vs Never RAG	Cost vs Always RAG	Gain, our protocol
Adaptive k (gap)	+0.029	+0.150	0.83x	−0.018
Adaptive RAG router	+0.052	+0.173	1.75x	−0.045
Confidence cascade	+0.083	+0.204	1.84x	+0.002
Closed-book gate	−0.069	+0.052	1.32x	−0.002
TSR planner	+0.054	+0.175	1.12x	+0.003

TABLE XV: The same 27 primary tests per plan set, the same policies and one test (pooled per-question test, Holm over 27), against three comparators: Always RAG and Never RAG (accuracy only, no cost matching), the best single fixed plan at or below the policy's cost, and the hull.

Plan set	Positive vs Always and Never RAG	Positive vs best single plan	Positive vs hull	Mean value of non adaptive mixing
single-step	23	5	0	+0.017
with two-hop	23	7	0	+0.023

TABLE XVI: Adaptive RAG as published: one T5 Large classifier per reader trained on the four pooled datasets, with the ladder no retrieval, BM25 with 5 passages and IRCoT. Gains on held-out folds over Always RAG (no cost matching) and over four hulls at its own cost: the global hull (here one plan, the reranker at depth 10), the label hull, the global hull with IRCoT as one more plan, and a random mix of its own three rungs. Single-step plan set (the two-hop set changes only the label hull); latency includes the classifier; Always RAG takes 0.4 to 1.8 s.

Reader	vs Always RAG	vs global hull	vs label hull	vs hull with IRCoT	vs rungs hull	Latency (s)	Share to IRCoT
Qwen2.5 7B	+0.070	−0.031	−0.040	−0.033	+0.005	14.2	0.58
Llama 3.1 8B	+0.071	−0.061	−0.058	−0.061	+0.002	12.1	0.59
Qwen3 14B	+0.091	−0.023	−0.029	−0.037	+0.016	40.5	0.53

($p = 0.733$). Adaptive k also has intervals below zero for Llama 3.1 8B and Gemma 3 27B, with adjusted p values from 0.314 to 0.565. So the table shows no hidden positive effect: every interval that excludes zero lies below it.

A null result does not show that the effect is zero [77], so we compare each one-sided 95% upper bound with the margin of 0.03. Under the per-question test, all 27 bounds lie below it with single-step plans (largest 0.029) and 26 of 27 with the two-hop plans (largest 0.032, the reader cascade to Qwen3 14B). The corrected test gives the same counts with smaller bounds (largest 0.025 and 0.031). With the corrected errors inflated by 2.61, still no primary test is positive; 42

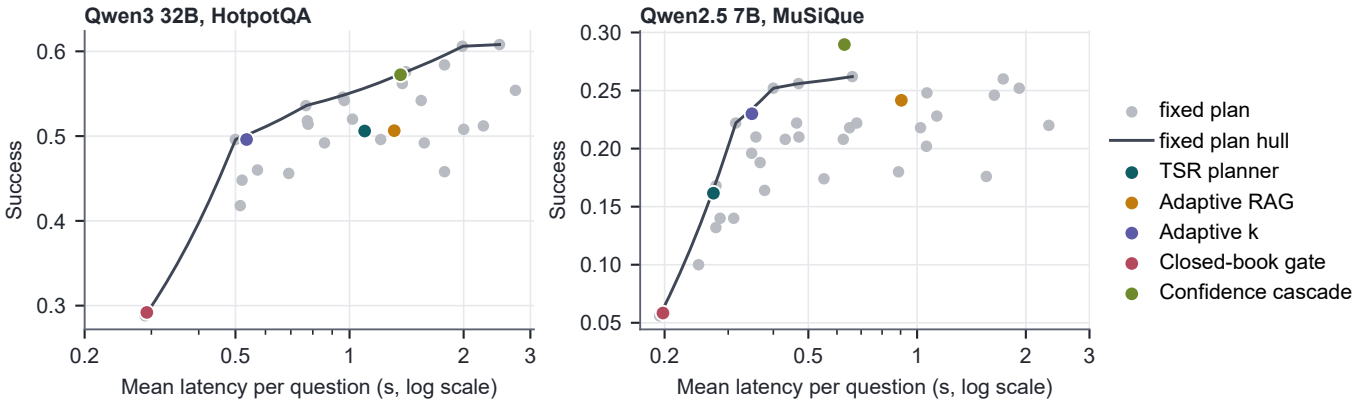


Fig. 6. Fixed plans, the fixed plan hull and the primary policies for two reader and dataset cells. The hull here is fitted on all 500 questions for illustration and is curved because a mix is linear in raw latency, not in log latency; the tests use hulls fitted on training folds and pool four datasets. Single cells are noisy: the confidence cascade lies above the line for Qwen2.5 7B on MuSiQue, yet its pooled gain is +0.013 and not significant.

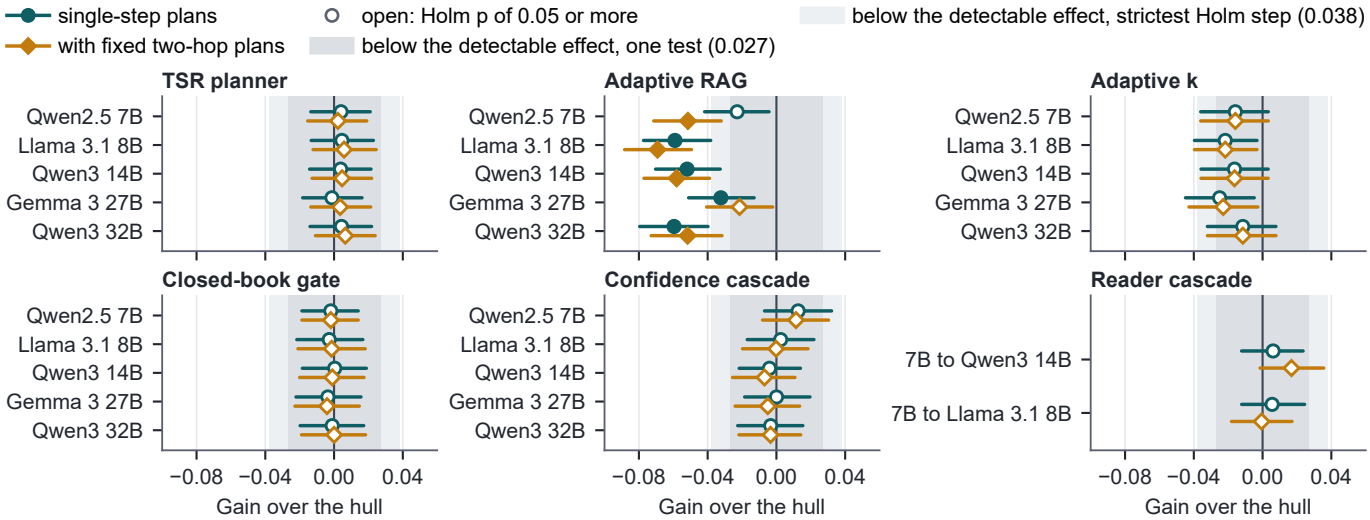


Fig. 7. H1: gain over the held-out fixed plan hull at equal mean latency, pooled over four datasets (2,000 questions), with 95% bootstrap intervals. Filled markers survive Holm's correction within the family of 27 tests; open markers do not. Grey bands mark gains below the minimum detectable effect (medians over tests): 0.027 for one test (dark) and 0.038 at the strictest Holm step (light).

of the 54 bounds stay below 0.03, 51 below 0.04 and 53 below 0.05 (largest 0.053, the same reader cascade; Tables XX and XXI). For the other 12 tests the five-seed refit bootstrap measures the spread directly (Table XIX): it is 0.68 to 1.64 times the corrected error, not 2.61, and 0.54 to 1.18 times the per-question error. Eleven of the 12 refit bounds lie below 0.03, the largest is 0.032 (the same reader cascade), and no refit test is significant (smallest p 0.068). So 53 of the 54 bounds exclude a gain of 0.03 under the per-question test, the corrected test and the refit bootstrap alike, and under each of them all 54 exclude 0.04; with Holm's correction over each plan set, the per-question test rejects a gain of 0.03 in 52 tests. The single-seed ratios that set the factor 2.61 therefore overstate the spread of our five-seed estimator, as expected.

Section VII shows that on audited labels the confidence cascade of Qwen2.5 7B excludes 0.04 but not 0.03. Table XX gives these counts for every family under the three variance estimates, and Table XXI lists every primary bound, so that readers can apply their own margin. Holm's correction adjusts p values, not bounds; the counts here are of unadjusted bounds. We also refit and retest the primary family on each half of every dataset separately, leaving out the two-reader cascades. On either half, none of the 25 tests per plan set is positive under the per-question or the corrected test; the largest gains are +0.008 to +0.018, and 22 to 24 of the 25 bounds lie below 0.03 and all below 0.05. The corrected test is narrower here because each fold gain compares the two arms on the same questions, which removes the spread in question difficulty that the unpaired per-question test keeps.

The refit bootstrap behind the inflation factor uses four tests: the confidence cascade of Qwen2.5 7B and the reader cascade to Qwen3 14B, which had the two largest primary bounds, and the per-plan router of Qwen3 32B in both plan sets, the secondary test closest to significance. Table XXIII gives it, and Table XXII compares the plain and the stratified bootstrap for every pooled test. Each resample draws the questions of

TABLE XVII: Gain over the held-out fixed plan hull at equal mean latency, pooled over 2,000 questions, for every policy and reader. S: single-step plans. T: with the fixed two-hop plans. Bold: Holm survivor within its family (27, 30 or 10 tests per plan set) with the 95% CI above or below zero. Reader cascades start from Qwen2.5 7B.

	Qwen2.5 7B		Llama 3.1 8B		Qwen3 14B		Gemma 3 27B		Qwen3 32B	
Policy	S	T	S	T	S	T	S	T	S	T
TSR planner	+0.004	+0.002	+0.005	+0.006	+0.004	+0.005	−0.001	+0.004	+0.004	+0.007
Adaptive RAG	−0.023	**−0.051**	**−0.059**	**−0.069**	**−0.052**	**−0.058**	**−0.032**	−0.021	**−0.060**	**−0.052**
Adaptive k	−0.016	−0.016	−0.022	−0.022	−0.016	−0.016	−0.025	−0.023	−0.012	−0.011
Closed-book gate	−0.002	−0.002	−0.003	−0.001	+0.000	−0.001	−0.004	−0.004	−0.001	+0.000
Confidence cascade	+0.013	+0.011	+0.002	+0.000	−0.004	−0.007	+0.000	−0.005	−0.003	−0.003
Reader cascade to Llama 3.1 8B	+0.006	−0.001								
Reader cascade to Qwen3 14B	+0.006	+0.017								
Adaptive k, published rule	−0.003	−0.007	+0.005	+0.005	−0.014	−0.023	+0.008	−0.016	−0.009	−0.018
Adaptive RAG, BGE M3 classifier	−0.020	**−0.031**	**−0.045**	**−0.057**	**−0.047**	**−0.045**	−0.029	−0.009	**−0.060**	**−0.045**
Adaptive RAG, FLAN-T5 classifier	−0.014	−0.015	**−0.037**	**−0.036**	**−0.037**	−0.022	−0.020	−0.001	**−0.042**	−0.018
Per-plan logistic regression	+0.013	+0.015	+0.004	+0.006	+0.022	+0.024	+0.020	+0.024	+0.029	**+0.030**
Per-plan kNN	+0.002	+0.004	−0.008	−0.005	+0.016	+0.019	+0.010	+0.017	+0.017	+0.023
METIS style router	−0.005	−0.003	+0.001	+0.003	−0.003	−0.004	+0.002	−0.006	−0.001	+0.013
Classifier gate	−0.002	−0.003	−0.001	−0.002	+0.000	−0.003	−0.001	−0.004	−0.001	−0.006
Planner, self-knowledge	−0.008	−0.007	−0.005	−0.004	+0.001	−0.002	−0.007	−0.008	−0.008	−0.014

TABLE XVIII: H1, primary family: gain over the held-out fixed plan hull at equal mean latency (27 pooled tests, 2,000 questions each).

Policy	Reader	Gain, single step [95% CI]	Holm p	Gain, with two hop [95% CI]	Holm p
TSR planner	Qwen2.5 7B	+0.004 [−0.014, +0.021]	1.000	+0.002 [−0.015, +0.019]	1.000
TSR planner	Llama 3.1 8B	+0.005 [−0.013, +0.023]	1.000	+0.006 [−0.012, +0.025]	1.000
TSR planner	Qwen3 14B	+0.004 [−0.014, +0.022]	1.000	+0.005 [−0.013, +0.022]	1.000
TSR planner	Gemma 3 27B	−0.001 [−0.018, +0.016]	1.000	+0.004 [−0.014, +0.021]	1.000
TSR planner	Qwen3 32B	+0.004 [−0.014, +0.022]	1.000	+0.007 [−0.011, +0.024]	1.000
Adaptive RAG router	Qwen2.5 7B	−0.023 [−0.042, −0.004]	0.382	−0.051 [−0.071, −0.032]	<0.001
Adaptive RAG router	Llama 3.1 8B	−0.059 [−0.077, −0.038]	<0.001	−0.069 [−0.088, −0.049]	<0.001
Adaptive RAG router	Qwen3 14B	−0.052 [−0.070, −0.033]	<0.001	−0.058 [−0.077, −0.039]	<0.001
Adaptive RAG router	Gemma 3 27B	−0.032 [−0.051, −0.013]	0.025	−0.021 [−0.041, −0.002]	0.733
Adaptive RAG router	Qwen3 32B	−0.060 [−0.080, −0.040]	<0.001	−0.052 [−0.073, −0.032]	<0.001
Adaptive k (gap)	Qwen2.5 7B	−0.016 [−0.036, +0.003]	1.000	−0.016 [−0.036, +0.003]	1.000
Adaptive k (gap)	Llama 3.1 8B	−0.022 [−0.040, −0.003]	0.539	−0.022 [−0.040, −0.003]	0.565
Adaptive k (gap)	Qwen3 14B	−0.016 [−0.036, +0.003]	1.000	−0.016 [−0.036, +0.003]	1.000
Adaptive k (gap)	Gemma 3 27B	−0.025 [−0.045, −0.005]	0.314	−0.023 [−0.043, −0.003]	0.545
Adaptive k (gap)	Qwen3 32B	−0.012 [−0.032, +0.008]	1.000	−0.011 [−0.032, +0.008]	1.000
Closed-book gate	Qwen2.5 7B	−0.002 [−0.019, +0.014]	1.000	−0.002 [−0.019, +0.014]	1.000
Closed-book gate	Llama 3.1 8B	−0.003 [−0.022, +0.017]	1.000	−0.001 [−0.021, +0.018]	1.000
Closed-book gate	Qwen3 14B	0.000 [−0.018, +0.019]	1.000	−0.001 [−0.020, +0.017]	1.000
Closed-book gate	Gemma 3 27B	−0.004 [−0.022, +0.016]	1.000	−0.004 [−0.023, +0.015]	1.000
Closed-book gate	Qwen3 32B	−0.001 [−0.020, +0.017]	1.000	0.000 [−0.019, +0.018]	1.000
Confidence cascade	Qwen2.5 7B	+0.013 [−0.007, +0.032]	1.000	+0.011 [−0.008, +0.030]	1.000
Confidence cascade	Llama 3.1 8B	+0.002 [−0.017, +0.022]	1.000	0.000 [−0.020, +0.018]	1.000
Confidence cascade	Qwen3 14B	−0.004 [−0.022, +0.014]	1.000	−0.007 [−0.026, +0.011]	1.000
Confidence cascade	Gemma 3 27B	0.000 [−0.018, +0.020]	1.000	−0.005 [−0.024, +0.014]	1.000
Confidence cascade	Qwen3 32B	−0.003 [−0.023, +0.015]	1.000	−0.003 [−0.022, +0.014]	1.000
Reader cascade	7B to Llama 3.1 8B	+0.006 [−0.012, +0.024]	1.000	−0.001 [−0.018, +0.017]	1.000
Reader cascade	7B to Qwen3 14B	+0.006 [−0.012, +0.024]	1.000	+0.017 [−0.001, +0.036]	1.000

each dataset with replacement, keeps the copies of a question in one fold, and reruns the fold split, the policy fitting, the hull vertices and the mixing weights; each uses one seed, so the interval also carries the variation over fold splits. Each resample holds only about 63% distinct questions, so a learned policy trains on less information and its refit mean shifts down: by 0.019 to 0.023 for the per-plan router, against about 0.001 for the cascades. We therefore read the refit spread around the observed gain. On these four tests its ratio to the corrected standard error is 1.42 to 1.64. Twelve more tests, drawn at random from the primary and secondary families with a fixed seed, give ratios of 0.96 to 1.47 to the per-question error and 1.03 to 2.61 to the corrected one. The largest, the confidence cascade of Qwen3 14B with the two-hop plans, sets the factor of 2.61; the refit bootstrap may itself overstate the spread, since it uses one seed per resample and duplicated questions.

Table XXIV splits the primary gains by dataset: Adaptive RAG loses most on NQ (-0.084) and Adaptive k on 2Wiki

TABLE XIX: The 12 primary tests whose bound under the corrected test with errors x2.61 exceeds 0.03, refitted with a bootstrap of 100 resamples and five seeds per resample: one-sided 95% upper bounds under each variance estimate, and the refit spread over the corrected standard error.

Plan set	Policy	Reader	Gain	Bound, per q.	Bound, corr.	Bound, x2.61	Bound, refit	Refit /corr. SE
single-step	Conf. cascade	Llama 3.1 8B	+0.002	0.019	0.014	0.032	0.021	1.64
single-step	Conf. cascade	Qwen2.5 7B	+0.013	0.029	0.025	0.045	0.029	1.36
single-step	TSR planner	Qwen2.5 7B	+0.004	0.019	0.016	0.036	0.012	0.68
single-step	Reader cascade	7B to Llama 3.1 8B	+0.006	0.021	0.017	0.035	0.024	1.63
single-step	Reader cascade	7B to Qwen3 14B	+0.006	0.021	0.017	0.035	0.022	1.48
with two-hop	TSR planner	Gemma 3 27B	+0.004	0.018	0.015	0.033	0.012	0.76
with two-hop	Conf. cascade	Qwen2.5 7B	+0.011	0.028	0.025	0.047	0.027	1.19
with two-hop	TSR planner	Qwen2.5 7B	+0.002	0.017	0.013	0.031	0.010	0.75
with two-hop	Reader cascade	7B to Llama 3.1 8B	−0.001	0.015	0.013	0.034	0.018	1.40
with two-hop	Reader cascade	7B to Qwen3 14B	+0.017	0.032	0.031	0.053	0.032	1.11
with two-hop	TSR planner	Qwen3 14B	+0.005	0.020	0.015	0.031	0.013	0.88
with two-hop	TSR planner	Qwen3 32B	+0.007	0.021	0.017	0.035	0.015	0.79

TABLE XX: Inference that accounts for refitting. Per question: t test on the 2,000 pooled gains, with the fitted policies and hulls fixed. Corrected: resampled t test on the 25 fold gains per reader and dataset (Nadeau and Bengio). ×2.61: corrected standard errors inflated by the largest ratio of the refit bootstrap spread to the corrected error. Positive: Holm p below 0.05 and interval above 0. Bound: one-sided 95% upper bound.

Family	Plan set	Tests	Positive: per question / corrected / ×2.61	Bound below 0.03: per question / corrected / ×2.61	Largest bound, corrected
Primary	single-step	27	0 / 0 / 0	27 / 27 / 22	0.025
Primary	with two-hop	27	0 / 0 / 0	26 / 26 / 20	0.031
Secondary, routers uncharged	single-step	30	0 / 2 / 0	25 / 26 / 20	0.042
Secondary, routers uncharged	with two-hop	30	1 / 3 / 0	23 / 24 / 15	0.042
Secondary, routers charged	single-step	30	0 / 0 / 0	30 / 30 / 22	0.023
Secondary, routers charged	with two-hop	30	0 / 0 / 0	30 / 30 / 17	0.028
Stronger	single-step	10	0 / 0 / 0	10 / 10 / 10	0.009
Stronger	with two-hop	10	0 / 0 / 0	10 / 10 / 10	0.006

(-0.069).

D. The Verdict Holds Across Costs and Grading

Cost axes. Latency depends on our hardware. Under the four costs that do not, no test is positive in either plan set, and the largest gain is $+0.016$ (Table VII). Calls give every depth of one retriever the same cost, so they cannot price depth; Adaptive k turns negative under calls for that reason.

Table XXV gives the positive and negative survivors and the largest gain under each cost. The calls axis has the most negative survivors, eight with the two-hop plans. Each row repeats the 27 primary tests with one cost in place of latency, so the hull and every policy are rebuilt for that cost. The columns give the positive and negative Holm survivors and the largest gain in each plan set. Negative survivors range from four (latency) to eight (calls, two-hop plans). The largest gain stays between $+0.006$ and $+0.017$ under every cost.

Grading. Under the strict rule the largest gain is $+0.010$, and under the lenient rule $+0.022$. Neither rule produces a positive test. Table XXVI lists the counts per rule and plan set: no row has a positive survivor, and every row has four or five negative ones. Here final is `label_final`, lenient accepts a strict match or a token F1 of at least 0.4, and strict accepts only exact or contained matches. The largest gain moves with the rule, from $+0.009$ (strict, two-hop plans) to $+0.022$ (lenient, two-hop plans). It stays below the margin of 0.03 under every rule.

A second judge from another model family, Mistral Small 3.1 24B at about 4 bits, regraded all 10,636 answers of the ambiguous band. The two judges agree on 87.3% of them (kappa 0.737). With its labels no single-step primary test is positive, and one two-hop test is (confidence cascade, Qwen2.5 7B, $+0.020$), which fails the rule that both plan sets must agree.

Cost accounting. Reading each policy and hull vertex at its realized per-question costs on the held-out fold, or computing plan costs from the training folds only, leaves no primary test positive; the largest gain is $+0.012$ to $+0.017$. Tail latency is where the policies do worse. At matched mean cost, the 95th percentile latency of a policy is a median 1.09 times that of its hull with single-step plans, and up to 1.68 times, because cascades and gates run two steps on some questions. Table XXVIII collects these checks for the primary family, and Table XXIX gives the tail ratio by policy. The TSR planner and the closed-book gate stay within 1.05 of their hull, and the

TABLE XXI: Every one-sided 95% upper bound of the primary family, under the per-question test and under the corrected test, single-step plans and with two-hop plans; readers can apply their own margin.

Policy	Reader	Gain, single	Bound, per question	Bound, corrected	Gain, two-hop	Bound, per question	Bound, corrected
TSR planner	Qwen2.5 7B	+0.004	0.019	0.016	+0.002	0.017	0.013
TSR planner	Llama 3.1 8B	+0.005	0.020	0.014	+0.006	0.021	0.014
TSR planner	Qwen3 14B	+0.004	0.019	0.013	+0.005	0.020	0.015
TSR planner	Gemma 3 27B	−0.001	0.014	0.010	+0.004	0.018	0.015
TSR planner	Qwen3 32B	+0.004	0.019	0.014	+0.007	0.021	0.017
Adaptive RAG router	Qwen2.5 7B	−0.023	−0.007	−0.008	−0.051	−0.035	−0.033
Adaptive RAG router	Llama 3.1 8B	−0.059	−0.043	−0.044	−0.069	−0.053	−0.052
Adaptive RAG router	Qwen3 14B	−0.052	−0.036	−0.034	−0.058	−0.042	−0.039
Adaptive RAG router	Gemma 3 27B	−0.032	−0.016	−0.016	−0.021	−0.005	−0.004
Adaptive RAG router	Qwen3 32B	−0.060	−0.043	−0.042	−0.052	−0.035	−0.029
Adaptive k (gap)	Qwen2.5 7B	−0.016	0.001	−0.005	−0.016	0.001	−0.005
Adaptive k (gap)	Llama 3.1 8B	−0.022	−0.006	−0.012	−0.022	−0.006	−0.012
Adaptive k (gap)	Qwen3 14B	−0.016	0.001	−0.004	−0.016	0.001	−0.004
Adaptive k (gap)	Gemma 3 27B	−0.025	−0.008	−0.012	−0.023	−0.006	−0.010
Adaptive k (gap)	Qwen3 32B	−0.012	0.006	−0.001	−0.011	0.006	−0.001
Closed-book gate	Qwen2.5 7B	−0.002	0.013	0.000	−0.002	0.013	0.000
Closed-book gate	Llama 3.1 8B	−0.003	0.014	0.004	−0.001	0.015	0.005
Closed-book gate	Qwen3 14B	0.000	0.017	0.002	−0.001	0.015	0.006
Closed-book gate	Gemma 3 27B	−0.004	0.012	0.000	−0.004	0.012	0.000
Closed-book gate	Qwen3 32B	−0.001	0.015	0.001	0.000	0.016	0.008
Confidence cascade	Qwen2.5 7B	+0.013	0.029	0.025	+0.011	0.028	0.025
Confidence cascade	Llama 3.1 8B	+0.002	0.019	0.014	0.000	0.016	0.011
Confidence cascade	Qwen3 14B	−0.004	0.011	0.000	−0.007	0.009	−0.001
Confidence cascade	Gemma 3 27B	0.000	0.016	0.006	−0.005	0.010	0.002
Confidence cascade	Qwen3 32B	−0.003	0.012	0.006	−0.003	0.012	0.007
Reader cascade	7B to Qwen3 14B	+0.006	0.021	0.017	+0.017	0.032	0.031
Reader cascade	7B to Llama 3.1 8B	+0.006	0.021	0.017	−0.001	0.015	0.013

TABLE XXII: Pooled tests under the plain bootstrap and a bootstrap stratified by dataset. Width ratio: stratified interval width over plain width.

Plan set	Family	Tests	Width ratio (median, range)	Interval above 0, plain / stratified	Upper bound below 0.03, plain / stratified
single-step	primary	27	1.00 (0.95 to 1.05)	0 / 0	26 / 26
single-step	secondary routers	30	0.99 (0.92 to 1.04)	3 / 3	24 / 24
with two-hop	primary	27	1.00 (0.93 to 1.05)	0 / 0	25 / 25
with two-hop	secondary routers	30	0.98 (0.96 to 1.04)	5 / 5	23 / 23

TABLE XXIII: Refit bootstrap for the tests closest to a decision: 100 resamples of the questions within each dataset, each rerunning folds, policy fitting, hull and weights (one seed per resample). The interval and p value use the refit spread (standard deviation) around the observed gain. Each resample holds only about 63% distinct questions, so a learned policy trains on less information and its refit mean shifts down (last column); the cascades, which learn little, do not shift.

Policy	Reader	Plan set	Gain	Plain 95% CI	Gain +/- 1.96 refit SD	p, refit SD	Refit mean shift
Per-plan LR	Qwen3 32B	single-step	+0.029	[+0.010, +0.048]	[+0.007, +0.051]	0.009	−0.023
Per-plan LR	Qwen3 32B	with two-hop	+0.030	[+0.011, +0.048]	[+0.007, +0.053]	0.011	−0.019
Confidence cascade	Qwen2.5 7B	with two-hop	+0.011	[−0.008, +0.030]	[−0.011, +0.034]	0.326	+0.001
Reader cascade	7B to Qwen3 14B	with two-hop	+0.017	[−0.001, +0.036]	[−0.007, +0.041]	0.172	−0.002

two cascades reach 1.29 to 1.42. We therefore also matched the hull to each policy at the 95th and the 99th percentile latency instead of the mean (Table XXVII). The hull is the best random mix of two fixed plans whose percentile on the training questions meets the target; the percentile of a mix is computed from the mixed latency distribution, not from the plans' percentiles. With the policy's percentile also measured on the training questions, and weights in steps of 0.01, no policy beats the hull under any test; the largest gain is $+0.020$, and policies lose under the per-question test in 7 to 9 tests per plan set (the reader cascades are left out, since their plans span two readers). Our first version took the policy's percentile

TABLE XXIV: Primary policies per dataset: gain over the hull on held-out questions with single-step plans, mean over the five readers (descriptive, no test). Train: the same gain measured on the training folds, where the policy and the hull were fitted (mean over datasets and readers).

	Held out					
Policy	NQ	Hotpot	2Wiki	MuSiQue	Pooled	Train
TSR planner	−0.004	−0.006	+0.023	+0.000	+0.003	+0.029
Adaptive RAG	−0.084	−0.040	−0.039	−0.017	−0.045	+0.072
Adaptive k	−0.010	+0.000	−0.069	+0.006	−0.018	−0.020
Closed-book gate	−0.002	−0.001	−0.002	−0.001	−0.002	+0.001
Confidence cascade	−0.006	+0.000	+0.007	+0.005	+0.002	+0.009

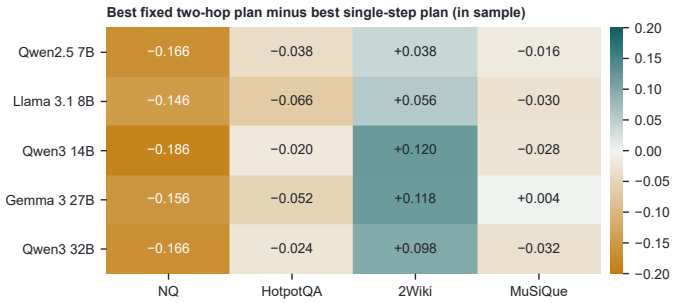


Fig. 8. Best fixed two-hop plan minus best single-step plan, per reader and dataset (in sample, `label_final`).

on the 100 test questions of each fold, with weights in steps of 0.05. There the TSR planner for Qwen3 14B passes the per-question test in both plan sets, at $+0.041$ and $+0.047$ (95th percentile) and $+0.058$ and $+0.063$ (99th), and three other policies pass at the 99th percentile in one or both plan sets; none passes the corrected test. A five-seed refit bootstrap of these tests confirms the 99th percentile gains of the TSR planner for Qwen3 14B (Holm p 0.013 and 0.002) and no other. But that gain comes mostly from folds where the noisy target, close to the slowest of 100 questions, lies below every retrieval plan's percentile on the training questions, so the hull falls back to closed book: in a quarter of its folds the hull is closed book alone and the gain is $+0.179$, against $+0.024$ in the other folds. Measured on the same questions for policy and hull, the tail result agrees with the mean: no policy beats the coin flip.

Two-hop plans. In sample, the best two-hop plan beats the best single-step plan on 2Wiki for all five readers, by 0.038 to 0.120, and loses on NQ for all five, by 0.146 to 0.186. Adding these plans to the hull and to every policy changes no primary verdict. Which kind of plan is best depends on the dataset, the pattern Section VI-F tests.

Table XXX names the best plan of each kind with its latency, and Fig. 8 maps the differences. Each row is one reader and dataset. The two middle columns give the best single-step plan and the best fixed two-hop plan, each with its success, its name (retriever and depth) and its mean latency; the last column is the difference. The best single-step plan changes with the dataset: Stella on NQ and the reranker on HotpotQA for every reader, and a mix of Stella, the reranker and RRF on 2Wiki and MuSiQue. On 2Wiki the best two-hop plan is BM25 with 3 passages per hop for all five readers. It is also faster there than the best single-step plan for four of the five readers; Qwen3 14B is the exception (4.55 s against 3.70 s). On HotpotQA the two-hop plan loses by 0.020 to 0.066. On MuSiQue it loses for four readers and wins by 0.004 for Gemma 3 27B.

E. Stronger and Secondary Policies

The secondary family tests whether better routers close the gap. Under the per-question test none is positive in both plan sets. One test is positive with the two-hop plans only: per-plan logistic regression on question embeddings for Qwen3 32B, at $+0.030$ (Holm $p = 0.042$), against $+0.029$ (Holm $p = 0.058$) with single-step plans. The corrected test finds this router positive in both plan sets for Qwen3 14B and Qwen3 32B, but with the 2.61 inflation it is positive nowhere, so it fails our rule. Both richer Adaptive RAG classifiers still lose: the FLAN-T5 classifier is significantly negative for one to three readers and the embedding classifier for three to four, depending on the plan set. The METIS style router stays within 0.013 of the hull, even with a free profiler.

Table XXXI gives all 30 secondary tests with intervals and Holm-adjusted p values, and Fig. 9 plots them. Per-plan k-nearest neighbours ranges from -0.008 to $+0.023$, with no survivor. The METIS style router ranges from -0.006 to $+0.013$, and Adaptive k with the published rule from -0.023 to $+0.008$. The two Adaptive RAG classifiers give six negative survivors with single-step plans and five with the two-hop plans. Each row is one method and reader, with the same columns as Table XVIII; Holm's correction runs over the 30 tests of each plan set. Intervals that lie fully above zero appear only for the per-plan routers. Logistic regression has them for Qwen3 14B, Gemma 3 27B and Qwen3 32B in both plan sets, and k-nearest neighbours for Qwen3 14B and Qwen3 32B with the two-hop plans. After the correction only logistic regression for Qwen3 32B with the two-hop plans remains ($p = 0.042$), and it is not significant with single-step plans ($p = 0.058$).

These router gains are measured before the router pays for its own signal. Its query embedding takes 24.6 ms per question when measured live, 6% to 10% of its replayed cost for Qwen2.5 7B. We charge this cost whenever the chosen plan does not embed the query with BGE M3 anyway, and the router tunes its cost weight with these costs. Then no embedding router is positive under either test in either plan set; the largest gain is $+0.014$, and all 60 corrected upper bounds lie below 0.03. Routers that predict the difference between two plans, with the pair and the threshold chosen on the training folds, do no better: with their own cost, 1 of 20 tests is positive, in one plan set only. Tuned for the best success within each of six budgets, the per-plan router gains $+0.016$ at the lowest budget before its own cost and -0.001 after. Across the six budgets and the five policies swept, the only interval above zero once costs are charged belongs to the reader cascade, at one of six budgets and without correction for the six.

TABLE XXV: Primary family under five cost axes: positive and negative Holm survivors.

Cost axis	Positive, single-step	Negative, single-step	Largest gain, single-step	Positive, two hop	Negative, two hop	Largest gain, two-hop
Latency (main)	0	4	+0.013	0	4	+0.017
Reader tokens	0	5	+0.012	0	5	+0.013
Tokens, output x4	0	5	+0.012	0	5	+0.014
FLOPs	0	5	+0.016	0	5	+0.011
Calls	0	5	+0.006	0	8	+0.008

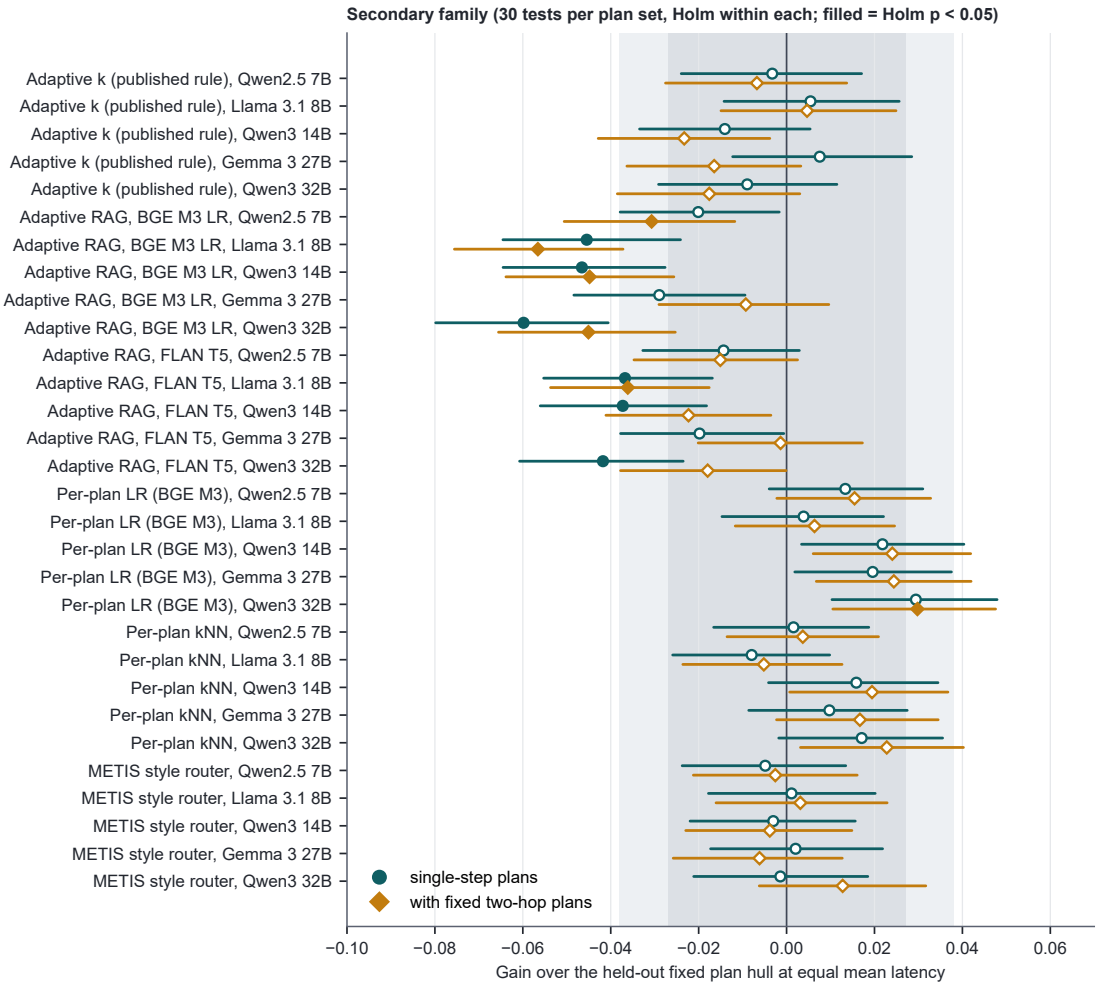


Fig. 9. Secondary family: gain over the held-out fixed plan hull at equal mean latency, with 95% bootstrap intervals, for 30 tests per plan set. Filled markers have Holm $p < 0.05$ within each plan set. Grey bands mark gains below the minimum detectable effect (medians over tests): 0.027 for one test (dark) and 0.038 at the strictest Holm step (light).

TABLE XXVI: H1 under three grading rules (27 tests per row).

Label rule	Plan set	Positive survivors	Negative survivors	Largest gain
final	single-step	0	4	+0.013
final	with two-hop	0	4	+0.017
lenient	single-step	0	4	+0.019
lenient	with two-hop	0	5	+0.022
strict	single-step	0	5	+0.010
strict	with two-hop	0	5	+0.009

Table XXXII gives every reader in both plan sets with and without the embedding cost under the per-question test; charging it lowers each gain by 0.008 to 0.022. Table XXXIII gives the difference routers. Before their own cost they gain $+0.007$ to $+0.037$ and are positive in 4 or 5 of 5 tests per plan set under the corrected test; with their cost only the classifier for Qwen3 14B stays positive, with the two-hop plans ($+0.027$). Table XXXIV gives the budget sweep. At the lowest budget most policies have no setting within the budget and run 1.02 to 2.02 times over it; they are still scored against the hull at their own cost.

The two stronger policies use five closed-book confidence

TABLE XXVII: Primary family against a hull matched at the policy's 95th or 99th percentile latency. The policy's percentile is measured either on the 100 test questions of each fold or on the training questions, where the hull's percentile is also measured (weights in steps of 0.05 and 0.01; the reader cascades are left out of the second version). Holm survivors with the interval above (Pos) or below (Neg) zero.

Policy percentile on	Plan set	Percentile	Tests	Pos/Neg, per q.	Pos/Neg, corr.	Max gain
test questions	single-step	P95	27	1/7	0/4	+0.041
test questions	single-step	P99	27	4/7	0/5	+0.058
test questions	with two-hop	P95	27	1/7	0/5	+0.047
test questions	with two-hop	P99	27	3/8	0/5	+0.063
training questions	single-step	P95	25	0/8	0/9	+0.019
training questions	single-step	P99	25	0/9	0/4	+0.020
training questions	with two-hop	P95	25	0/7	0/8	+0.017
training questions	with two-hop	P99	25	0/9	0/6	+0.018

TABLE XXVIII: The primary family under each check (corrected test; Holm within each plan set). Closed-book prompt: Llama 3.1 8B rerun with the closed-book answers of the few-shot chain-of-thought prompt that Moskvoretskii et al. [10] use, at their measured time and at the old time (Holm over the policies replayed). The closed-book rows replay 7 policies: 5 primary and 2 secondary.

Check (primary family)	Positive, single-step	Largest gain, single-step	Positive, with two-hop	Largest gain, with two-hop
Paper protocol, corrected test	0 of 27	+0.013	0 of 27	+0.017
Realized per-question costs	0 of 27	+0.013	0 of 27	+0.016
Plan costs from training folds only	0 of 27	+0.012	0 of 27	+0.017
Closed-book prompt of [10], measured time (Llama, 7 policies)	0 of 7	+0.005	0 of 7	+0.005
Closed-book prompt of [10], old time (Llama, 7 policies)	0 of 7	+0.004	0 of 7	+0.005
Labels from the outside judge	0 of 27	+0.015	1 of 27	+0.020

TABLE XXIX: Tail latency at matched mean cost: 95th percentile of the per-question latency of each policy over that of its hull (the hull's plan drawn per question with its weight), median over readers, seed 0.

Policy	p95 ratio, single-step	p95 ratio, with two-hop
TSR planner	1.02	1.02
Adaptive RAG router	1.11	1.07
Adaptive k (gap)	1.09	1.09
Closed-book gate	1.01	1.05
Confidence cascade	1.29	1.32
Reader cascade	1.36	1.42
Per-plan LR	1.14	1.09

features, the kind of plain uncertainty signal that often matches complex adaptive pipelines [10]. All 10 tests are null, with gains from -0.014 to $+0.001$. The planner with these features approaches the hull as training data grows, from -0.039 at 20% of the training folds to -0.008 at 100% for Qwen2.5 7B, but stays below zero for four of the five readers. Table XXXV gives the curve for every reader. More training data moves it toward the hull but not clearly past it. The embedding router also gains with data (before its embedding cost): pooled over readers, from -0.002 at 20% of the training folds to $+0.018$ at 100%, and it is still rising at 400 training questions per dataset; Section VI-F tests larger pooled training sets. More data from other datasets does not help, though: one router trained on the training folds of all four datasets (about 1,600 questions, with dataset indicators) gains -0.003 to $+0.018$ before its own cost, less than per-dataset training for four of the five readers, and no test survives Holm's correction. Which plan suits a question seems to depend on the dataset, so pooling adds questions but not the right signal. Table XXXVI gives its curve for each reader. It rises for every reader, most for Qwen3 32B (from $+0.000$ to $+0.029$) and least for Llama 3.1 8B (from -0.010 to $+0.004$), and unlike the planner with confidence features it has not flattened at 100%.

Table XXXVII gives the 10 tests of the stronger family with intervals. Its rows are the classifier gate (c1) and the planner with self-knowledge features (c2) for each reader, with the same columns as Table XVIII. All 20 intervals contain zero, and every adjusted p value is 1.000. The classifier gate stays between -0.006 and 0.000, and the planner with confidence features between -0.014 and $+0.001$. Table XXXVIII lists the learning curve as written by the analysis script, and Fig. 10 plots it. Each row is one reader, sorted by name, and each column trains the planner on that share of the training folds. The values are those of Table XXXV. For four of the five readers the largest step comes from 20% to 40% of the data; after 60% every change is 0.008 or smaller. The curves rise with more data and flatten near zero.

TABLE XXX: Fixed two-hop plans against the best single-step plan (in sample).

Reader	Dataset	Best single step	Best two-hop	Difference
Qwen2.5 7B	NQ	0.730 (stella_1_5b@5, 0.51 s)	0.564 (bm25_2hop@5, 1.10 s)	−0.166
Qwen2.5 7B	HotpotQA	0.530 (qwen3_reranker_0_6b@3, 1.41 s)	0.492 (bm25_2hop@5, 1.01 s)	−0.038
Qwen2.5 7B	2Wiki	0.406 (bge_bm25_rrf@20, 1.37 s)	0.444 (bm25_2hop@3, 0.97 s)	+0.038
Qwen2.5 7B	MuSiQue	0.262 (stella_1_5b@10, 0.66 s)	0.246 (bm25_2hop@3, 0.98 s)	−0.016
Llama 3.1 8B	NQ	0.698 (stella_1_5b@5, 0.57 s)	0.552 (bm25_2hop@5, 1.54 s)	−0.146
Llama 3.1 8B	HotpotQA	0.588 (qwen3_reranker_0_6b@10, 1.71 s)	0.522 (bm25_2hop@3, 1.25 s)	−0.066
Llama 3.1 8B	2Wiki	0.406 (qwen3_reranker_0_6b@10, 2.20 s)	0.462 (bm25_2hop@3, 1.24 s)	+0.056
Llama 3.1 8B	MuSiQue	0.228 (qwen3_reranker_0_6b@5, 1.87 s)	0.198 (bm25_2hop@5, 1.65 s)	−0.030
Qwen3 14B	NQ	0.776 (stella_1_5b@10, 2.36 s)	0.590 (bm25_2hop@5, 5.00 s)	−0.186
Qwen3 14B	HotpotQA	0.596 (qwen3_reranker_0_6b@3, 2.62 s)	0.576 (bm25_2hop@5, 4.63 s)	−0.020
Qwen3 14B	2Wiki	0.434 (qwen3_reranker_0_6b@10, 3.70 s)	0.554 (bm25_2hop@3, 4.55 s)	+0.120
Qwen3 14B	MuSiQue	0.286 (qwen3_reranker_0_6b@10, 3.71 s)	0.258 (bm25_2hop@5, 5.92 s)	−0.028
Gemma 3 27B	NQ	0.726 (stella_1_5b@10, 1.31 s)	0.570 (bm25_2hop@5, 1.75 s)	−0.156
Gemma 3 27B	HotpotQA	0.612 (qwen3_reranker_0_6b@10, 2.15 s)	0.560 (bm25_2hop@5, 1.47 s)	−0.052
Gemma 3 27B	2Wiki	0.388 (qwen3_reranker_0_6b@10, 2.83 s)	0.506 (bm25_2hop@3, 1.39 s)	+0.118
Gemma 3 27B	MuSiQue	0.284 (stella_1_5b@10, 1.37 s)	0.288 (bm25_2hop@3, 1.40 s)	+0.004
Qwen3 32B	NQ	0.760 (stella_1_5b@10, 1.75 s)	0.594 (bm25_2hop@5, 2.39 s)	−0.166
Qwen3 32B	HotpotQA	0.608 (qwen3_reranker_0_6b@10, 2.49 s)	0.584 (bm25_2hop@5, 1.94 s)	−0.024
Qwen3 32B	2Wiki	0.502 (bge_bm25_rrf@10, 1.86 s)	0.600 (bm25_2hop@3, 1.83 s)	+0.098
Qwen3 32B	MuSiQue	0.344 (stella_1_5b@10, 2.10 s)	0.312 (bm25_2hop@5, 3.00 s)	−0.032

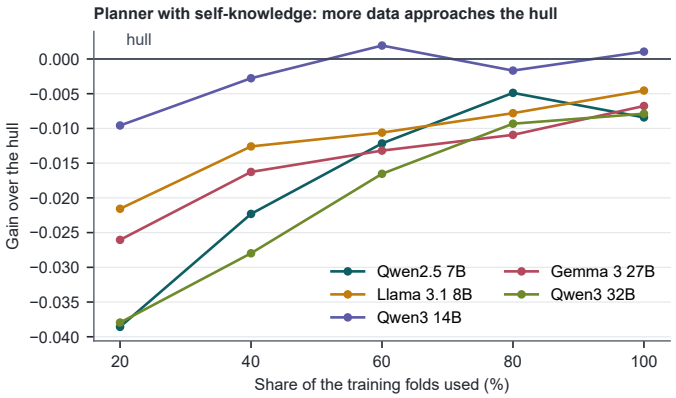


Fig. 10. Learning curve of the planner with self-knowledge features: pooled gain over the hull (single-step plans) when it is trained on a share of the training folds.

F. A Mixed Workload

So far the policy and the hull are fitted for one reader and one dataset, so the hull knows the dataset. A deployed system may serve several kinds of questions at once. We therefore pool the 2,000 questions of each reader; each question keeps its own corpus and stored answers. Folds run across the pool, and one hull and each policy are fitted on the pooled training folds with no dataset indicator. We compare each policy with two baselines. The global hull is the best mix of fixed plans for the whole pool; it is the baseline a deployed system can use. The label hull is the best mix of fixed plans for each dataset under one shared budget; it gets each question's dataset for free, so it is a reference, not a baseline. The gain over the global hull then splits into a workload part, the label hull minus the global hull at the policy's cost, and the gain over the label hull, which is what the policy knows about a question beyond its dataset. Table XXXIX gives the results of all three tests.

The largest held-out gap between the label hull and the global hull, over a grid of budgets, is 0.009 to 0.027 with single-step plans and 0.027 to 0.048 with the two-hop plans. We read these maxima on held-out folds, so they describe the room for a policy that only recognises the workload; they are not tests. No primary policy beats either hull. Against the global hull they are significantly negative in 5 tests with single-step plans and in 3 with the two-hop plans, and Adaptive RAG trained on the pool with its silver labels loses in 9 of its 10 tests. Before their own cost, the embedding routers beat the global hull in 6 and 5 tests under the corrected test, and the k-nearest neighbour router for Qwen3 14B gains most ($+0.053$), but with errors inflated by 2.61 none survives Holm's correction. With their own cost they gain up to $+0.024$ with single-step plans and $+0.042$ with the two-hop plans, and none passes all three tests in both plan sets. Almost all of the gain is the workload part. For per-plan logistic regression with single-step plans, the mean gain of $+0.029$ is $+0.033$ from the workload and -0.004 from the question. A classifier that only guesses the dataset from the question embedding and then runs the label hull's mix does as well: with its own cost it gains $+0.003$ to $+0.022$ with single-step plans and $+0.027$ to $+0.042$ with the two-hop plans, positive under the corrected test for all five readers with the two-hop plans and for none with single-step plans. It guesses 79% of held-out questions right; benchmark datasets differ in style, so real mixed traffic may be harder to recognise. Against the label hull, no policy of any family is positive; the largest gain is $+0.006$. Table XL gives the split for every policy. The workload part is $+0.019$ to $+0.039$, and the gain over the label hull is negative for every policy, from -0.024 to -0.002. Charging the router's own cost can even raise its gain over the global hull (Gemma 3 27B, two-hop plans: $+0.038$ to $+0.042$), because the router

TABLE XXXI: Secondary family: stronger routers and the METIS style router (30 tests per plan set).

Method	Reader	Gain, single step	Holm p	Gain, with two hop	Holm p
Adaptive k (published rule)	Qwen2.5 7B	−0.003 [−0.024, +0.017]	1.000	−0.007 [−0.028, +0.014]	1.000
Adaptive k (published rule)	Llama 3.1 8B	+0.005 [−0.014, +0.026]	1.000	+0.005 [−0.015, +0.025]	1.000
Adaptive k (published rule)	Qwen3 14B	−0.014 [−0.033, +0.005]	1.000	−0.023 [−0.043, −0.004]	0.427
Adaptive k (published rule)	Gemma 3 27B	+0.008 [−0.012, +0.028]	1.000	−0.016 [−0.036, +0.003]	1.000
Adaptive k (published rule)	Qwen3 32B	−0.009 [−0.029, +0.011]	1.000	−0.018 [−0.038, +0.003]	1.000
Adaptive RAG, BGE M3 LR	Qwen2.5 7B	−0.020 [−0.038, −0.002]	0.723	−0.031 [−0.051, −0.012]	0.045
Adaptive RAG, BGE M3 LR	Llama 3.1 8B	−0.045 [−0.065, −0.024]	<0.001	−0.057 [−0.076, −0.037]	<0.001
Adaptive RAG, BGE M3 LR	Qwen3 14B	−0.047 [−0.064, −0.028]	<0.001	−0.045 [−0.064, −0.026]	<0.001
Adaptive RAG, BGE M3 LR	Gemma 3 27B	−0.029 [−0.048, −0.009]	0.073	−0.009 [−0.029, +0.010]	1.000
Adaptive RAG, BGE M3 LR	Qwen3 32B	−0.060 [−0.080, −0.041]	<0.001	−0.045 [−0.066, −0.025]	<0.001
Adaptive RAG, FLAN T5	Qwen2.5 7B	−0.014 [−0.033, +0.003]	1.000	−0.015 [−0.035, +0.002]	1.000
Adaptive RAG, FLAN T5	Llama 3.1 8B	−0.037 [−0.055, −0.017]	0.003	−0.036 [−0.054, −0.018]	0.004
Adaptive RAG, FLAN T5	Qwen3 14B	−0.037 [−0.056, −0.018]	0.002	−0.022 [−0.041, −0.004]	0.427
Adaptive RAG, FLAN T5	Gemma 3 27B	−0.020 [−0.038, −0.001]	0.723	−0.001 [−0.020, +0.017]	1.000
Adaptive RAG, FLAN T5	Qwen3 32B	−0.042 [−0.061, −0.024]	<0.001	−0.018 [−0.038, 0.000]	1.000
Per-plan LR	Qwen2.5 7B	+0.013 [−0.004, +0.031]	1.000	+0.015 [−0.002, +0.033]	1.000
Per-plan LR	Llama 3.1 8B	+0.004 [−0.015, +0.022]	1.000	+0.006 [−0.012, +0.025]	1.000
Per-plan LR	Qwen3 14B	+0.022 [+0.003, +0.040]	0.415	+0.024 [+0.006, +0.042]	0.200
Per-plan LR	Gemma 3 27B	+0.020 [+0.002, +0.037]	0.642	+0.024 [+0.007, +0.042]	0.184
Per-plan LR	Qwen3 32B	+0.029 [+0.010, +0.048]	0.058	+0.030 [+0.011, +0.048]	0.042
Per-plan kNN	Qwen2.5 7B	+0.002 [−0.017, +0.019]	1.000	+0.004 [−0.014, +0.021]	1.000
Per-plan kNN	Llama 3.1 8B	−0.008 [−0.026, +0.010]	1.000	−0.005 [−0.024, +0.013]	1.000
Per-plan kNN	Qwen3 14B	+0.016 [−0.004, +0.034]	1.000	+0.019 [+0.001, +0.037]	0.704
Per-plan kNN	Gemma 3 27B	+0.010 [−0.009, +0.027]	1.000	+0.017 [−0.002, +0.034]	1.000
Per-plan kNN	Qwen3 32B	+0.017 [−0.002, +0.035]	1.000	+0.023 [+0.003, +0.040]	0.326
METIS style router	Qwen2.5 7B	−0.005 [−0.024, +0.013]	1.000	−0.003 [−0.021, +0.016]	1.000
METIS style router	Llama 3.1 8B	+0.001 [−0.018, +0.020]	1.000	+0.003 [−0.016, +0.023]	1.000
METIS style router	Qwen3 14B	−0.003 [−0.022, +0.016]	1.000	−0.004 [−0.023, +0.015]	1.000
METIS style router	Gemma 3 27B	+0.002 [−0.017, +0.022]	1.000	−0.006 [−0.026, +0.013]	1.000
METIS style router	Qwen3 32B	−0.001 [−0.021, +0.018]	1.000	+0.013 [−0.006, +0.032]	1.000

TABLE XXXII: Per-plan logistic regression without and with its own cost: 24.6 ms for the query embedding and classifiers, charged when the chosen plan does not embed the query with BGE-M3 anyway; lambda is tuned with these costs. Per dataset: trained on 400 questions per dataset (Holm p: secondary family of 30 tests). Pooled training (exploratory): one model on the training folds of all four datasets. p: unadjusted.

Reader	Plan set	Per dataset, replay [95% CI]	Holm p	Per dataset, own cost [95% CI]	p	Pooled, replay	p	Pooled, own cost	p
Qwen2.5 7B	single-step	+0.013 [−0.004, +0.031]	1.000	+0.002 [−0.016, +0.019]	0.841	−0.002	0.807	−0.012	0.185
Llama 3.1 8B	single-step	+0.004 [−0.015, +0.022]	1.000	−0.006 [−0.023, +0.012]	0.521	+0.005	0.614	−0.010	0.249
Qwen3 14B	single-step	+0.022 [+0.003, +0.040]	0.415	+0.011 [−0.007, +0.030]	0.223	+0.011	0.237	0.000	0.998
Gemma 3 27B	single-step	+0.020 [+0.002, +0.037]	0.642	+0.005 [−0.012, +0.023]	0.542	+0.010	0.278	0.000	0.995
Qwen3 32B	single-step	+0.029 [+0.010, +0.048]	0.058	+0.007 [−0.011, +0.025]	0.419	+0.012	0.180	−0.011	0.219
Qwen2.5 7B	with two-hop	+0.015 [−0.002, +0.033]	1.000	+0.007 [−0.010, +0.024]	0.430	−0.003	0.721	−0.008	0.368
Llama 3.1 8B	with two-hop	+0.006 [−0.012, +0.025]	1.000	−0.002 [−0.020, +0.016]	0.812	+0.007	0.425	−0.006	0.505
Qwen3 14B	with two-hop	+0.024 [+0.006, +0.042]	0.200	+0.013 [−0.005, +0.031]	0.154	+0.016	0.086	+0.007	0.479
Gemma 3 27B	with two-hop	+0.024 [+0.007, +0.042]	0.184	+0.014 [−0.003, +0.032]	0.114	+0.018	0.046	+0.016	0.088
Qwen3 32B	with two-hop	+0.030 [+0.011, +0.048]	0.042	+0.013 [−0.006, +0.030]	0.154	+0.012	0.177	−0.002	0.788

retunes its cost weight and moves to another point of a steep hull.

More pooled training data does not change this. Trained on 320 to 1,600 pooled questions, the routers' gain over the label hull rises from about -0.02 and levels off from 1,280 questions, at -0.004 and -0.003 with single-step plans before their own cost; it does not turn positive. Table XLI gives the curve. Against the global hull the per-plan logistic regression router gains $+0.011$ at 320 questions and about $+0.03$ from 1,280 questions on (mean over readers and plan sets), so more data mainly teaches the workload part.

Pooling changes the task as well as the amount of data, so this is only indirect evidence on the 400 training questions of the per-dataset tests.

G. Multi-Step Retrieval Wins Through Its Extra Steps

IRCoT is the one method that beats the hull (Fig. 11). Against the single-step hull the per-question test finds 4 wins in 9 tests. Three are on 2Wiki, one per reader, with gains of $+0.088$ to $+0.156$; the fourth is Qwen3 14B on MuSiQue at

TABLE XXXIII: Routers trained on the difference between two plans (BGE M3 features, pair and threshold chosen on the training folds), without and with their own 24.6 ms cost. Pair AUROC: held-out questions where the two chosen plans disagree, against the per-plan router's score on the same questions and pairs.

Router	Plan set	Gain, no own cost	Positive	Gain, own cost	Positive	Pair AUROC	Per-plan LR, same pairs
Difference classifier	single-step	+0.010 to +0.025	4	+0.002 to +0.016	0	0.674	0.677
Difference classifier	with two-hop	+0.013 to +0.037	5	+0.010 to +0.027	1	0.683	0.686
Difference regression	single-step	+0.008 to +0.028	4	+0.001 to +0.016	0	0.647	0.656
Difference regression	with two-hop	+0.007 to +0.028	4	+0.004 to +0.018	0	0.657	0.665

TABLE XXXIV: Budget sweep (single-step plans, three seeds): each policy tuned on the training folds to its best success within the budget, scored against the held-out hull at its mean cost. Budgets from the cheapest plan (1) to the most successful plan (6); mean gain over readers. Last column: reader and budget tests with the corrected interval above 0 (no correction for the six budgets). At budget 1, 22 of 26 policy and reader runs have no setting within the budget and cost 1.02 to 2.02 times the budget; they are still scored against the hull at their own cost.

Policy	1	2	3	4	5	6	Intervals above 0
TSR planner	−0.013	−0.002	0.000	−0.002	−0.001	0.000	0
Closed-book gate	−0.034	−0.089	−0.073	−0.061	−0.041	−0.032	0
Confidence cascade	−0.034	−0.019	−0.014	−0.006	−0.007	−0.006	0
Reader cascade (7B to Qwen3 14B)	−0.159	−0.003	+0.012	+0.015	+0.010	+0.012	1
Per-plan LR, no own cost	+0.016	+0.010	+0.003	+0.001	−0.001	−0.001	5
Per-plan LR, own cost	−0.001	+0.003	−0.001	−0.001	−0.002	−0.001	0

TABLE XXXV: Learning curve of the planner with self-knowledge features: pooled gain over the hull (single-step plans) when it is trained on a share of the training folds.

Reader	20%	40%	60%	80%	100%
Qwen2.5 7B	−0.039	−0.022	−0.012	−0.005	−0.008
Llama 3.1 8B	−0.022	−0.013	−0.011	−0.008	−0.005
Qwen3 14B	−0.010	−0.003	+0.002	−0.002	+0.001
Gemma 3 27B	−0.026	−0.016	−0.013	−0.011	−0.007
Qwen3 32B	−0.038	−0.028	−0.017	−0.009	−0.008

TABLE XXXVI: Learning curve of per-plan logistic regression: pooled gain over the hull when trained on a share of the training folds (single-step plans, without its embedding cost).

Reader	20%	40%	60%	80%	100%
Qwen2.5 7B	−0.005	+0.002	+0.004	+0.010	+0.013
Llama 3.1 8B	−0.010	−0.007	−0.005	−0.002	+0.004
Qwen3 14B	+0.005	+0.008	+0.016	+0.015	+0.022
Gemma 3 27B	−0.001	+0.008	+0.005	+0.014	+0.020
Qwen3 32B	0.000	+0.006	+0.011	+0.023	+0.029
Pooled	−0.002	+0.003	+0.006	+0.012	+0.018

$+0.074$. A refit bootstrap of all 18 IRCoT tests has a spread of 0.95 to 1.57 times the per-question error; it keeps three wins and drops Llama 3.1 8B on 2Wiki (Holm $p = 0.073$). The 2Wiki wins hold under all three grading rules. Across rules, the strict rule adds a win for Llama 3.1 8B on MuSiQue, and the lenient rule drops Qwen3 14B on MuSiQue. Table XLII shows each test under every rule.

Table XLIII gives the nine tests with their intervals, the Holm-adjusted p values with and without the two-hop plans in the hull, and the verdict under each grading rule. IRCoT does not win on HotpotQA for any reader. Each row is one reader and dataset. The columns give IRCoT's success, its latency relative to the best fixed plan, the gain over the single-step hull with its interval and Holm p, the gain and Holm p when the two fixed hop plans join the hull, and the verdict under `label_final`, lenient and strict grading. A yes marks a Holm survivor with the interval above zero. All three HotpotQA intervals span zero, and Llama 3.1 8B on HotpotQA is the only negative estimate (-0.034).

The wins are expensive: IRCoT runs 11 to 22 times longer than the best fixed plan, with 5.4 to 6.4 reader calls and 3.6 to 3.9 retrievals per question on 2Wiki. It needs 18.6 to 79.8 seconds per question, against about 0.7 to 3.7 seconds for the best fixed plan. On HotpotQA it is within 0.034 of the best fixed plan for all three readers.

Table XLV adds the strict accuracy and the stopping behaviour of every run. Its columns are success under `label_final` and strict grading, reader calls and retrieval

TABLE XXXVII: Classifier gate and planner with closed-book confidence features against the hull.

Policy	Reader	Gain, single step [95% CI]	Holm p	Gain, with two hop [95% CI]	Holm p
Classifier gate (c1)	Qwen2.5 7B	−0.002 [−0.018, +0.012]	1.000	−0.003 [−0.019, +0.012]	1.000
Classifier gate (c1)	Llama 3.1 8B	−0.001 [−0.020, +0.018]	1.000	−0.002 [−0.022, +0.017]	1.000
Classifier gate (c1)	Qwen3 14B	0.000 [−0.018, +0.018]	1.000	−0.003 [−0.021, +0.014]	1.000
Classifier gate (c1)	Gemma 3 27B	−0.001 [−0.019, +0.018]	1.000	−0.004 [−0.022, +0.013]	1.000
Classifier gate (c1)	Qwen3 32B	−0.001 [−0.018, +0.016]	1.000	−0.006 [−0.022, +0.011]	1.000
Planner with self-knowledge (c2)	Qwen2.5 7B	−0.008 [−0.026, +0.009]	1.000	−0.007 [−0.025, +0.010]	1.000
Planner with self-knowledge (c2)	Llama 3.1 8B	−0.005 [−0.023, +0.015]	1.000	−0.004 [−0.023, +0.016]	1.000
Planner with self-knowledge (c2)	Qwen3 14B	+0.001 [−0.018, +0.020]	1.000	−0.002 [−0.021, +0.017]	1.000
Planner with self-knowledge (c2)	Gemma 3 27B	−0.007 [−0.024, +0.011]	1.000	−0.008 [−0.025, +0.010]	1.000
Planner with self-knowledge (c2)	Qwen3 32B	−0.008 [−0.026, +0.010]	1.000	−0.014 [−0.032, +0.003]	1.000

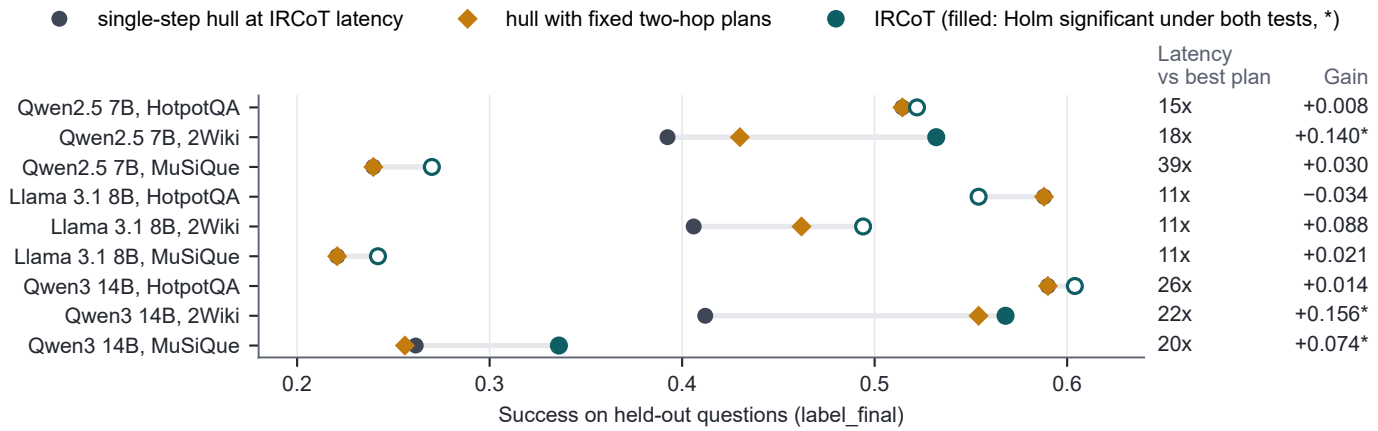


Fig. 11. H2: IRCoT against the held-out hull at IRCoT's own mean latency. Filled markers and stars mark tests that survive Holm's correction over the 9 tests under the per-question test and the refit bootstrap. Latency is IRCoT's mean latency divided by that of the best fixed plan (18 to 22 times for the three wins, up to 39 times elsewhere); Gain is measured against the single-step hull.

TABLE XXXVIII: Gain over the hull as the training share grows (descriptive).

Reader	20% of training	40% of training	60% of training	80% of training	100% of training
Gemma 3 27B	−0.026	−0.016	−0.013	−0.011	−0.007
Llama 3.1 8B	−0.022	−0.013	−0.011	−0.008	−0.005
Qwen2.5 7B	−0.039	−0.022	−0.012	−0.005	−0.008
Qwen3 14B	−0.010	−0.003	+0.002	−0.002	+0.001
Qwen3 32B	−0.038	−0.028	−0.017	−0.009	−0.008

calls per question, the share of questions on which IRCoT stopped with an answer, seconds per question, and the number of questions whose demonstrations were dropped. Qwen2.5 7B stops on an answer least often (0.746 to 0.872) and makes the most reader calls (5.74 to 6.73). IRCoT stops on an answer in 75% to 97% of questions; the rest stop at the step or paragraph limit. Under the strict rule its success falls by 0.074 to 0.080 on HotpotQA but changes by at most 0.014 on 2Wiki. For Qwen3 14B on 2Wiki the run dropped the demonstrations for 17 questions; no other run dropped any.

A fixed second retrieval explains part of the gain. Once the two fixed hop plans join the hull, IRCoT keeps 2 wins, both under the refit bootstrap too: Qwen2.5 7B on 2Wiki (+0.102) and Qwen3 14B on MuSiQue (+0.080). For Qwen3 14B on 2Wiki its gain falls from +0.156 to +0.014: a fixed plan that always asks one follow-up query recovers almost all of it, at a cost known in advance. IRCoT also makes more retrieval and reader calls than any fixed plan (Table XLIV).

Does IRCoT gain from deciding when to stop, or only from running more steps? We therefore built the fixed chains F_T of Section IV: IRCoT's own loop with its stopping rule turned off, run for exactly $T = 1$ to 4 retrievals; the number of steps is fixed in advance, and each query is still the reader's newest reasoning sentence. We rebuild them from the stored IRCoT chains, since with greedy decoding the shared first steps are identical; a replay of IRCoT from these chains reproduces all 6,000 of its answers. Steps beyond IRCoT's own are run; a reused step is charged the question's measured time per call, within a median of 1% to 4% of a rerun. Table XLVI gives each chain on each dataset. Averaged over readers, F_3 on 2Wiki reaches 0.531 at 19.1 s, as IRCoT does at 42.8 s, and F_4 on HotpotQA and MuSiQue is within 0.007 of IRCoT at 68% to 76% of its latency; on NQ, F_3 reaches 0.575 at 19.4 s against 0.570 at 38.5 s. Accuracy rises from F_1 to F_3 on every dataset and changes by at most 0.021 from F_3 to F_4. On the same questions, IRCoT minus F_4 is −0.006 to +0.010 for every reader and dataset, every interval contains zero, and F_4 is 1.3 to 25.0 s faster. IRCoT is slower because after its

TABLE XXXIX: Mixed workload: the 2,000 questions of each reader pooled, one hull and each policy fitted on the pooled training folds, no dataset indicator. Global hull: best mix of fixed plans for the pool; this is the baseline a deployed system can use. Label hull: best mix of fixed plans per dataset under one budget; it knows each question's dataset and is a reference only. Holm within each family and comparator. Pos.: tests with the interval above 0 after Holm under the per-question test, the corrected test, and the corrected test with errors $\times 2.61$. Neg. and the label hull columns use the corrected test.

Policies	Plan set	Tests	Pos., global: per q. /corr. /$\times 2.61$	Neg., global	Gain, global hull	Pos./neg., label hull	Gain, label hull
Primary policies	single-step	27	0 / 0 / 0	5	-0.077 to $+0.011$	0 / 8	-0.098 to $+0.003$
Primary policies	with two-hop	27	0 / 0 / 0	3	-0.061 to $+0.021$	0 / 10	-0.092 to 0.000
Adaptive RAG, silver labels	single-step	5	0 / 0 / 0	5	-0.073 to -0.027	0 / 5	-0.094 to -0.038
Adaptive RAG, silver labels	with two-hop	5	0 / 0 / 0	4	-0.052 to -0.004	0 / 5	-0.082 to -0.041
Secondary routers, no own cost	single-step	25	5 / 6 / 0	5	-0.083 to $+0.053$	0 / 5	-0.103 to $+0.006$
Secondary routers, no own cost	with two-hop	25	5 / 5 / 0	2	-0.048 to $+0.053$	0 / 8	-0.084 to $+0.004$
Embedding routers, own cost	single-step	10	0 / 1 / 0	0	-0.003 to $+0.024$	0 / 1	-0.026 to -0.005
Embedding routers, own cost	with two-hop	10	6 / 4 / 0	0	$+0.013$ to $+0.042$	0 / 0	-0.014 to -0.003
Dataset guess, own cost	single-step	5	0 / 0 / 0	0	$+0.003$ to $+0.022$	0 / 0	-0.023 to -0.001
Dataset guess, own cost	with two-hop	5	5 / 5 / 0	0	$+0.027$ to $+0.042$	0 / 0	-0.004 to 0.000

TABLE XL: Where the gain on mixed traffic comes from (mean over readers): gain over the global hull = workload part (label hull minus global hull at the policy's cost) + gain over the label hull.

Policy	Global, single	Workload, single	Label, single	Global, two hop	Workload, two hop	Label, two-hop
TSR planner	+0.006	+0.026	-0.020	+0.007	+0.030	-0.024
Reader cascade	+0.006	+0.019	-0.012	+0.020	+0.033	-0.013
Per-plan LR	+0.029	+0.033	-0.004	+0.031	+0.037	-0.006
Per-plan kNN	+0.032	+0.035	-0.003	+0.031	+0.036	-0.005
Per-plan LR, own cost	+0.007	+0.027	-0.020	+0.031	+0.038	-0.007
Per-plan kNN, own cost	+0.014	+0.028	-0.015	+0.026	+0.036	-0.011
Dataset guess + label hull	+0.029	+0.036	-0.006	+0.035	+0.039	-0.004
Dataset guess, own cost	+0.012	+0.021	-0.009	+0.036	+0.038	-0.002

TABLE XLI: Learning curve on the mixed workload: gain over the label hull, which removes the workload part, mean over readers, by the number of pooled training questions.

Plan set	Policy	320	640	960	1,280	1,600
single-step	Per-plan LR	-0.016	-0.010	-0.006	-0.005	-0.004
single-step	Per-plan LR, own cost	-0.023	-0.022	-0.019	-0.020	-0.020
single-step	Per-plan kNN	-0.019	-0.013	-0.007	-0.002	-0.003
single-step	Per-plan kNN, own cost	-0.028	-0.024	-0.019	-0.016	-0.015
with two-hop	Per-plan LR	-0.018	-0.013	-0.008	-0.006	-0.006
with two-hop	Per-plan LR, own cost	-0.024	-0.016	-0.012	-0.009	-0.007
with two-hop	Per-plan kNN	-0.022	-0.016	-0.010	-0.005	-0.005
with two-hop	Per-plan kNN, own cost	-0.027	-0.020	-0.015	-0.012	-0.011

last retrieval it keeps reasoning: on average it makes 5.1 to 5.7 reader calls for 3.3 to 3.7 retrievals, so 1.6 to 2.0 calls add no passages, while F_T answers right after its last retrieval. Once the chains join the hull, IRCoT wins none of 12 tests over the four datasets, under the per-question test and the refit bootstrap, with or without the two-hop plans. Its largest gain is $+0.034$ (Qwen2.5 7B on MuSiQue). On NQ it loses to the single-step hull by 0.14 to 0.17 for every reader; these three losses have the smallest p values, so adding NQ to the family leaves Holm's adjusted p for every other test unchanged, and the three wins of Fig. 11 stand. So on these readers IRCoT's stopping rule adds nothing; its wins come from extra steps with generated queries, and chains with a fixed query rule would be needed to test those queries.

A variant of Adaptive RAG uses the same ladder of no retrieval, one retrieval and IRCoT, but is trained only on the three multi-hop datasets, with the two classifiers of Section IV and our silver labels. It sends 39% to 80% of questions to IRCoT and costs 8 to 52 s per question. Pooled per reader, it beats neither the fixed plan hull nor a hull that may also run IRCoT at random (corrected test). Its one positive cell, Qwen3 14B on 2Wiki with single-step plans ($+0.097$ and $+0.100$ for its two classifiers), is IRCoT's own gain there, and it turns negative once the two-hop plans join the hull. Llama 3.1 8B on HotpotQA loses in every comparison, by 0.087 to 0.095. Table XLVII gives the pooled tests and the counts per reader and dataset cell. Against a hull with IRCoT as one more plan, its pooled gains run from -0.059 to $+0.005$: routing among

TABLE XLII: IRCoT gain over the held-out hull at its own latency under three grading rules. Bold: Holm survivor over the 9 tests with the 95% CI above zero. Last column: hull with the fixed two-hop plans, `label_final`.

		Single-step hull			Two-hop
Reader	Data	Final	Lenient	Strict	Final
Qwen2.5 7B	HotpotQA	+0.008	−0.004	+0.008	+0.008
Qwen2.5 7B	2Wiki	**+0.140**	**+0.119**	**+0.136**	**+0.102**
Qwen2.5 7B	MuSiQue	+0.030	+0.025	+0.024	+0.030
Llama 3.1 8B	HotpotQA	−0.034	−0.036	−0.028	−0.034
Llama 3.1 8B	2Wiki	**+0.088**	**+0.109**	**+0.104**	+0.032
Llama 3.1 8B	MuSiQue	+0.021	+0.029	**+0.048**	+0.021
Qwen3 14B	HotpotQA	+0.014	+0.010	+0.031	+0.014
Qwen3 14B	2Wiki	**+0.156**	**+0.127**	**+0.126**	+0.014
Qwen3 14B	MuSiQue	**+0.074**	+0.050	**+0.059**	**+0.080**

TABLE XLIII: H2: IRCoT against the held-out fixed plan hull at IRCoT's own mean latency. Holm p: per-question test. Holm p, refit: refit bootstrap (100 resamples of the questions, hull refitted each time).

Reader	Dataset	IRCoT success	Latency vs best plan	Gain, single step hull	Holm p	Holm p, refit	Gain, with two hop	Holm p	Holm p, refit	Sig. final/ len./strict (y/n)
Qwen2.5 7B	HotpotQA	0.522	15x	+0.008	1.000	1.000	+0.008	1.000	1.000	n/n/n
Qwen2.5 7B	2Wiki	0.532	18x	+0.140	<0.001	<0.001	+0.102	<0.001	<0.001	y/y/y
Qwen2.5 7B	MuSiQue	0.270	39x	+0.030	0.627	1.000	+0.030	0.878	1.000	n/n/n
Llama 3.1 8B	HotpotQA	0.554	11x	−0.034	0.627	1.000	−0.034	0.878	1.000	n/n/n
Llama 3.1 8B	2Wiki	0.494	11x	+0.088	<0.001	0.073	+0.032	0.878	1.000	y/y/y
Llama 3.1 8B	MuSiQue	0.242	11x	+0.021	0.807	1.000	+0.021	1.000	1.000	n/n/y
Qwen3 14B	HotpotQA	0.604	26x	+0.014	1.000	1.000	+0.014	1.000	1.000	n/n/n
Qwen3 14B	2Wiki	0.568	22x	+0.156	<0.001	<0.001	+0.014	1.000	1.000	y/y/y
Qwen3 14B	MuSiQue	0.336	20x	+0.074	0.003	0.019	+0.080	0.001	0.023	y/n/y

TABLE XLIV: IRCoT runs. Succ.: IRCoT success; Best: best fixed plan (in sample). LM and Ret.: reader and retrieval calls per question. Sec.: seconds per question. Lat.: latency relative to the best fixed plan.

Reader	Data	Succ.	Best	LM	Ret.	Sec.	Lat.
Qwen2.5 7B	HotpotQA	0.522	0.530	5.7	3.6	21.2	15×
Qwen2.5 7B	2Wiki	0.532	0.406	6.4	3.9	24.7	18×
Qwen2.5 7B	MuSiQue	0.270	0.262	6.7	3.8	26.1	39×
Llama 3.1 8B	HotpotQA	0.554	0.588	4.5	3.2	18.6	11×
Llama 3.1 8B	2Wiki	0.494	0.406	5.4	3.7	24.1	11×
Llama 3.1 8B	MuSiQue	0.242	0.228	4.7	3.4	19.7	11×
Qwen3 14B	HotpotQA	0.604	0.596	4.9	3.4	68.3	26×
Qwen3 14B	2Wiki	0.568	0.434	5.4	3.6	79.8	22×
Qwen3 14B	MuSiQue	0.336	0.286	5.3	3.6	75.9	20×

TABLE XLV: IRCoT runs: accuracy, calls, stopping and latency.

Reader	Dataset	label_final	strict	LM calls	Retrieval calls	Stops on answer	Seconds per question	Demos dropped (questions)
Qwen2.5 7B	2Wiki	0.532	0.518	6.39	3.86	0.828	24.7	0
Qwen2.5 7B	HotpotQA	0.522	0.442	5.74	3.60	0.872	21.2	0
Qwen2.5 7B	MuSiQue	0.270	0.220	6.73	3.80	0.746	26.1	0
Llama 3.1 8B	2Wiki	0.494	0.498	5.42	3.67	0.970	24.1	0
Llama 3.1 8B	HotpotQA	0.554	0.480	4.52	3.25	0.972	18.6	0
Llama 3.1 8B	MuSiQue	0.242	0.214	4.72	3.42	0.972	19.7	0
Qwen3 14B	2Wiki	0.568	0.562	5.42	3.64	0.974	79.8	17
Qwen3 14B	HotpotQA	0.604	0.528	4.88	3.42	0.966	68.3	0
Qwen3 14B	MuSiQue	0.336	0.284	5.29	3.56	0.938	75.9	0

the three rungs adds nothing over a random mix of the same rungs at the same cost.

VII. Analysis and Discussion

Q1. Is there anything to gain? Yes. We pick two plans on the training folds that together answer the most questions. An oracle that knows, for each held-out question, which of the two answers it and runs the cheaper correct one beats the hull

TABLE XLVI: Fixed step chains F_T (IRCoT's reasoning for exactly T retrievals, stopping off) against IRCoT: success and mean latency in seconds, mean over the three IRCoT readers. Last column: IRCoT minus F_4 on the same questions, range over readers.

Dataset	F_1	F_2	F_3	F_4	IRCoT	IRCoT minus F_4
NQ	0.470 (0.9)	0.550 (10.2)	0.575 (19.4)	0.574 (28.5)	0.570 (38.5)	−0.006 to −0.002
HotpotQA	0.433 (0.8)	0.512 (9.7)	0.555 (18.6)	0.560 (27.4)	0.560 (36.0)	−0.002 to +0.004
2Wiki	0.349 (0.7)	0.485 (9.9)	0.531 (19.1)	0.530 (28.3)	0.531 (42.8)	−0.002 to +0.004
MuSiQue	0.150 (0.9)	0.206 (9.9)	0.255 (18.7)	0.276 (27.7)	0.283 (40.5)	+0.002 to +0.010

TABLE XLVII: Adaptive RAG with the published ladder (closed-book, single-step plan, IRCoT) on the three multi-hop datasets, pooled per reader (Qwen2.5 7B, Llama 3.1 8B, Qwen3 14B). Fixed hull: mix of fixed plans. Hull with IRCoT: IRCoT added as one more plan. Corrected test. Pooled: Holm over the three readers. Cells: the nine reader and dataset tests, Holm over nine.

Classifier	Plan set	IRCoT share	Gain, fixed hull	Pooled pos./neg.	Cells pos./neg.	Gain, hull with IRCoT	Pooled pos./neg.	Cells pos./neg.
TF IDF LR	single-step	0.53 to 0.60	−0.030 to +0.029	0 / 0	1 / 1	−0.049 to +0.005	0 / 1	0 / 1
TF IDF LR	with two-hop	0.53 to 0.60	−0.049 to +0.016	0 / 1	0 / 1	−0.059 to 0.000	0 / 2	0 / 2
BGE M3 LR	single-step	0.51 to 0.56	−0.025 to +0.017	0 / 0	1 / 1	−0.043 to −0.007	0 / 1	0 / 1
BGE M3 LR	with two-hop	0.51 to 0.56	−0.043 to +0.004	0 / 1	0 / 1	−0.053 to −0.012	0 / 2	0 / 1

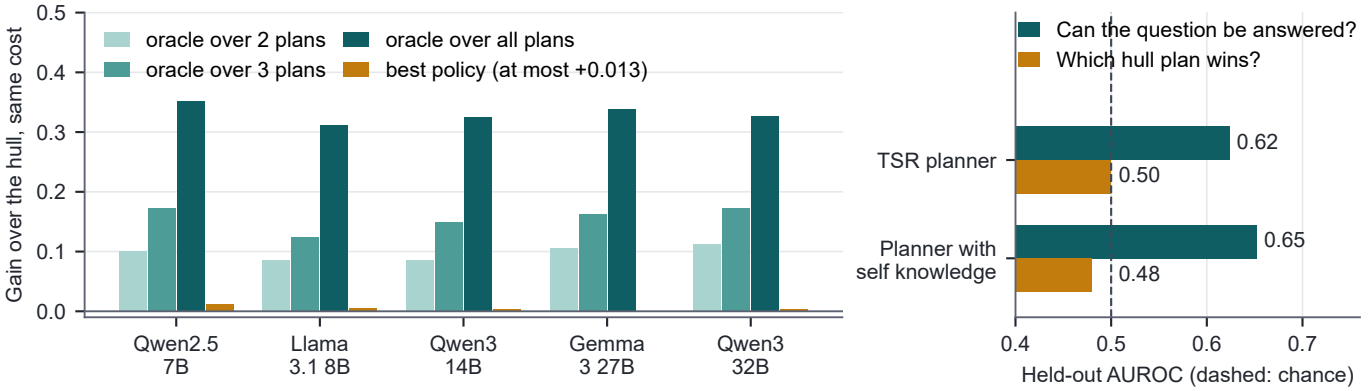


Fig. 12. Left: held-out headroom. An oracle that knows which of 2, 3 or all fixed plans answers each question, and runs the cheapest one that does, compared with the hull at the oracle's own mean latency; the amber bar is the best primary policy. Right: held-out AUROC of the two planners for whether a plan answers a question (teal), and for which of the two hull plans answers when exactly one does (amber).

by +0.098 at the same mean latency (Fig. 12, left). The gap is +0.085 to +0.112 across readers, +0.156 with three plans and +0.331 with all plans. Under the strict rule fewer questions are answered by any plan (0.627 to 0.720 against 0.706 to 0.758, Table XLVIII), so part of this headroom rests on the judge. The oracle still picks the correct plan from one noisy label per held-out question, so it overstates what any policy could reach [39]; it is a diagnostic upper bound, not an achievable gain. Read at each policy's own mean cost, an oracle frontier over all plan pairs, chosen on the training folds, lies 0.025 to 0.160 above the hull; the best primary policy closes at most 13% of that gap, and the embedding router up to 18% before its own cost. The null result is not a ceiling effect.

Table XLIX gives the full version: the oracle gap with two, three and all plans, the share of questions that any plan answers under both rules, and the largest share of the two plan gap that a policy closes. That share is 0.13 for Qwen2.5 7B and at most 0.05 for the other readers. Each row is one reader. The gap grows with the number of plans the oracle may choose from, from +0.085 to +0.112 with two plans to +0.312 to +0.351 with all plans. Under strict grading the share of questions that any plan answers falls by 0.038 to 0.093, so the oracle numbers depend on the grader.

Table XLIX compares each oracle with the hull at the oracle's own cost, and divides gains measured at the policy's cost by gaps measured at the oracle's cost. Table L removes that mismatch. For every pair of plans we compute the oracle that runs the cheaper plan that answers; the pairs on the upper envelope of these oracles, chosen on the training folds, form an oracle frontier that we read at each policy's own mean cost. The gap is smallest for the closed-book gate (0.025 to 0.086), which runs at low cost where every plan is cheap, and largest for Adaptive k and the per-plan router (up to 0.160). Adaptive RAG loses 20% to 77% of the gap, the TSR planner closes at most 5%, and the confidence cascade at most 13%, for Qwen2.5 7B. Before its embedding cost, the per-plan router closes 5% to 18%, the most of any policy.

Q2. Why do policies miss it? The planners predict whether each plan will answer a question with held-out AUROC 0.62 (TSR planner) and 0.65 (with self-knowledge features). Yet

TABLE XLVIII: Held-out headroom at matched cost, mean over datasets. Gap: oracle over 2, 3 or all fixed plans minus the hull at the oracle's mean latency. Any: share of questions that some plan answers, under `label_final` and the strict rule. Closed: largest share of the 2 plan gap closed by a primary policy.

	Gap			Any		
Reader	2	3	All	Final	Strict	Closed
Qwen2.5 7B	+0.100	+0.173	+0.351	0.717	0.647	0.13
Llama 3.1 8B	+0.085	+0.125	+0.312	0.706	0.655	0.05
Qwen3 14B	+0.086	+0.149	+0.326	0.720	0.663	0.04
Gemma 3 27B	+0.106	+0.163	+0.338	0.720	0.627	0.00
Qwen3 32B	+0.112	+0.172	+0.327	0.758	0.720	0.04

TABLE XLIX: Held-out oracle gap over the hull at matched cost, and the share policies close.

Reader	Oracle gap, 2 plans	3 plans	All plans	Any plan correct, label_final	Any plan correct, strict	Largest share of the 2 plan gap closed by a policy
Qwen2.5 7B	+0.100	+0.173	+0.351	0.717	0.647	0.13
Llama 3.1 8B	+0.085	+0.125	+0.312	0.706	0.655	0.05
Qwen3 14B	+0.086	+0.149	+0.326	0.720	0.663	0.04
Gemma 3 27B	+0.106	+0.163	+0.338	0.720	0.627	0.00
Qwen3 32B	+0.112	+0.172	+0.327	0.758	0.720	0.04

TABLE L: Oracle headroom at each policy's own mean cost: gap of the pair oracle frontier over the hull (pairs chosen on the training folds), and the fraction of that gap the policy closes (pooled gain over gap), mean over datasets. Per-plan LR without its embedding cost.

Policy	Qwen2.5 7B	Llama 3.1 8B	Qwen3 14B	Gemma 3 27B	Qwen3 32B
TSR planner	0.113 / +0.04	0.087 / +0.05	0.104 / +0.04	0.101 / −0.01	0.132 / +0.03
Adaptive RAG router	0.117 / −0.20	0.077 / −0.77	0.095 / −0.55	0.104 / −0.31	0.101 / −0.59
Adaptive k (gap)	0.108 / −0.15	0.110 / −0.20	0.119 / −0.14	0.124 / −0.20	0.138 / −0.08
Closed-book gate	0.043 / −0.04	0.060 / −0.05	0.025 / +0.02	0.086 / −0.04	0.025 / −0.04
Confidence cascade	0.098 / +0.13	0.069 / +0.04	0.080 / −0.05	0.095 / +0.00	0.090 / −0.04
Per-plan LR	0.120 / +0.11	0.080 / +0.05	0.143 / +0.15	0.138 / +0.14	0.160 / +0.18

their predictions for different plans of the same question correlate at 0.99. On questions that exactly one of the two hull plans answers, their AUROC for which plan it is falls to 0.50 and 0.48, which is chance (Fig. 12, right, and Table LI). Over all five seeds they are 0.51 and 0.48; for seed 0 the 95% intervals are $[0.47, 0.53]$ and $[0.45, 0.51]$, on 1,842 and 1,595 such questions. The planner with self-knowledge features also sends 61% of questions to a single plan. The embedding router differs: its predictions across plans correlate at 0.74, and its AUROC on such questions is 0.60 (seed 0: $[0.57, 0.63]$, 2,397 questions), lowest for Llama 3.1 8B (0.53), the reader where it gains least.

So our two TSR planners learn which questions are hard, not which plan suits a question. A difficulty signal alone does not help. A question that is hard for every plan gains nothing from a costly plan, and an easy one is answered by the cheap plan. Value lies only in the questions where plans disagree, and there the planners guess. Such questions are few: the two plans of a hull disagree, with one answering and the other not, on a mean of 16% to 24% of questions, and two retrievers at depth 5 on 18%. Picking the arg max over noisy estimates of near equal plans also makes the chosen plan look better on the training folds than it is [52]. The embedding routers do learn something about plans. The routers that predict the difference between two plans separate the pairs they choose with AUROC 0.65 to 0.69, and the per-plan router reaches 0.66 to 0.69 on the same questions and pairs, so a direct target does not help. Before their own cost, the cheaper plan of the chosen pair is closed-book in 60% of cases with single-step plans and 46% with the two-hop plans, so much of what they learn is whether the reader already knows the answer. That edge is real but too small to pay for their own embedding.

Table LII gives the same quantities to three decimals: per-plan AUROC 0.652 and 0.623, AUROC for which hull plan wins 0.480 and 0.499, and most used plan share 0.613 and 0.506. Per-plan AUROC measures how well the predicted success separates answered from unanswered questions, plan by plan. Which hull plan wins is the same measure on the questions that exactly one of the two hull plans answers. Most used plan share is the share of questions sent to the planner's most frequent plan, and Judge share is the share of answers that the judge graded, for the policy and for its hull.

Table LII uses the folds of seed 0 only and gives no intervals. Table LIII repeats the analysis over all five seeds and adds the per-plan router. The pair AUROC is computed within each reader and dataset block and then averaged, so it does not reward a router for knowing which plan is better on a whole dataset. Both planners stay near chance in every seed

TABLE LI: What the planners learn (held out, seed 0 folds, single-step plans, mean over 20 cells). Plan AUC: AUROC of predicted success per plan. Corr.: correlation of predictions across plans. Pair AUC: AUROC for which hull plan answers when exactly one does. Top plan: share of questions sent to the most used plan. Judge: share of answers graded by the judge, policy / hull. Self knowledge: the planner with closed-book confidence features.

Policy	Plan AUC	Corr.	Pair AUC	Top plan	Judge
TSR planner	0.62	0.99	0.50	0.51	0.152 / 0.152
Self knowledge	0.65	0.99	0.48	0.61	0.157 / 0.158

TABLE LII: What the planners learn, and grading exposure of policy and hull.

Policy	Per-plan AUROC	Which hull plan wins, AUROC	Most used plan share	Judge share, policy /hull
Planner with self-knowledge	0.652	0.480	0.613	0.157 / 0.158
TSR planner	0.623	0.499	0.506	0.152 / 0.152

(0.50 to 0.54 and 0.47 to 0.49), on 1,842 and 1,595 discordant questions per seed. The per-plan router reaches 0.60, with a seed 0 interval of $[0.57, 0.63]$, and its predictions across plans correlate at 0.74 rather than 0.99: it ranks plans differently for different questions. It also spreads its choices more: its most used plan takes 43% of questions, against 50% and 60% for the planners.

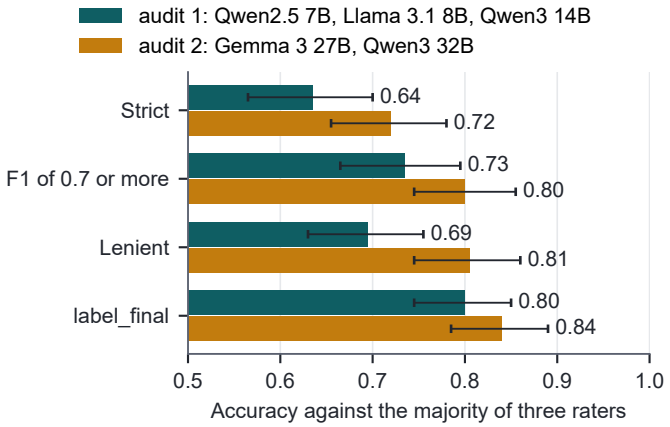


Fig. 13. Accuracy of four grading rules against the majority of three human raters, with 95% intervals, on two audits of 200 answers each.

Q3. Can the grading explain the null? Grading error matters here only if it differs between the policy and the hull. First, `label_final` agrees with the human majority on 0.80 and 0.84 of audited answers, against 0.64 and 0.72 for the strict rule (Fig. 13). The audits oversample ambiguous answers: 71% of audited items lie in the ambiguous band, against 15% of all answers. Its accuracy is 0.77 inside the band and 0.95 outside it, so weighted to all answers it is 0.90 and 0.93 on the two audits, and 0.92 $[0.88, 0.95]$ together. By reader, its specificity is 0.92 to 0.96, so the judge rarely accepts a wrong answer from any reader. Second, both arms meet the judge equally often. Over all 67 tests in each plan set, the share of answers that the judge graded differs between a policy and its hull mix by a median of 0.0004 (range -0.030 to $+0.022$). Equal exposure does not prove equal error rates, since two policies can make different kinds of mistakes. If the error rates were equal, a true difference would shrink by J = sensitivity + specificity $-$ 1, here 0.60 and 0.68, and an observed 0.03 would mean 0.044 to 0.050; we give this only as a sensitivity calculation under that untested assumption. Tables LIV and LV give the exposure of every test and the agreement by reader. Sensitivity varies more than specificity, from 0.52 for Qwen2.5 7B to 0.84 for Qwen3 32B, on 38 to 132 audited answers per reader.

Third, every test compares two arms of the same reader, so a judge that favours one reader's answers favours both arms alike. This matters because the judge is a Qwen3 model: compared with the outside judge, it accepts a share of Qwen3 32B's ambiguous answers that is higher by 0.055. The reader cascades mix two readers, one of them Qwen3 14B; the audit below covers that case. The outside judge is no closer to the human majority (J of 0.574 against 0.613) and leaves the verdict unchanged (Section VI-D). Table LVI compares the two judges with the human majority on the 393 audited answers of the ambiguous band. For the other readers, the Qwen3 judge accepts between 0.044 less and 0.034 more than the outside judge, and for Qwen3 14B 0.007 less. With the outside judge's labels, gains change by a median of -0.0002 to $+0.0009$ per family.

Fourth, we audited the answers on which a policy and its hull disagree, where a difference in error rates would act. For three tests near a decision (the confidence cascade of Qwen2.5 7B, the reader cascade to Qwen3 14B and the per-plan router of Qwen3 32B), 27% to 40% of questions are discordant, and we sampled 100 of them per test, 25 per dataset. Two raters who are not authors labelled the 586 distinct answers, blind to the test, the arm, the reader and every automatic label, with written guidelines. They agreed on 94.0% of them (kappa 0.862) and settled the other 35 together. Against these labels, `label_final` errs on 4% to 7% of answers in each arm, almost always by marking a correct answer wrong. If anything it favours the hull, which works against a null claim: it shifts the gain by -0.011 $[-0.024, 0.000]$, -0.008 $[-0.032, +0.012]$ and 0.000 $[-0.019, +0.022]$. A second round with the same raters and guidelines sampled 97 more discordant questions of the confidence cascade, all that one dataset had left, and

TABLE LIII: What the policies learn, over all five seeds (single-step plans, held out). Pair AUROC: for questions that exactly one of the two hull plans answers, AUROC for which one, averaged over the 20 reader and dataset blocks; interval: bootstrap over questions within blocks, seed 0.

Policy	Plan AUROC	Cross plan corr.	Pair AUROC	Seed range	Seed 0 95% CI	Pair questions per seed	Top plan share
TSR planner	0.624	0.994	0.514	0.499 to 0.538	[0.466, 0.528]	1,842	0.50
Planner, self-knowledge	0.654	0.993	0.481	0.473 to 0.490	[0.450, 0.508]	1,595	0.60
Per-plan LR	0.647	0.735	0.596	0.582 to 0.609	[0.573, 0.629]	2,397	0.43

TABLE LIV: Grading exposure of every policy against its hull mix (mean over folds, seeds and datasets): share of answers whose label came from the judge, and share of long contained answers.

Plan set	Tests	Judge share, policy minus hull: median	Range	Long contained, policy minus hull: range
single-step	67	+0.0004	−0.025 to +0.022	−0.029 to +0.009
with two-hop	67	+0.0004	−0.030 to +0.021	−0.036 to +0.007

TABLE LV: label_final against the majority of three human raters, by the reader that wrote the answer. label_final is the string rules, with the Qwen3 14B judge on the ambiguous band. Each reader was audited in one audit only. The Qwen3 14B row is graded by a judge of the same model.

Reader	Audit	Answers	Agreement	Sensitivity	Specificity	J
Gemma 3 27B	2	132	0.833	0.721	0.953	0.674
Llama 3.1 8B	1	94	0.798	0.714	0.921	0.635
Qwen2.5 7B	1	68	0.765	0.517	0.949	0.466
Qwen3 14B	1	38	0.868	0.750	0.955	0.705
Qwen3 32B	2	128	0.875	0.836	0.918	0.754

TABLE LVI: The judge in our pipeline (Qwen3 14B) and a judge from outside every reader family (Mistral Small 3.1 24B, about 4 bits) against the human majority, on the audited answers of the ambiguous band.

Judge	Answers	Agreement	Sensitivity	Specificity	J
Qwen3 14B	393	0.791	0.683	0.930	0.613
Mistral Small 3.1 24B	393	0.768	0.638	0.936	0.574

50 of IRCoT for Qwen3 14B on 2Wiki. The raters agreed on 97.1% of the 278 new answers (kappa 0.929) and settled the other 8 together; the settled label agreed with rater B on 5 of them. Audit 4 alone gives a shift of 0.000 $[-0.017, +0.017]$ for the cascade. On all 197 questions the shift is -0.005 $[-0.015, +0.004]$; without it the gain is $+0.018$, still not significant, with upper bounds of 0.035 (per-question) and 0.031 (corrected), so the test excludes 0.04 but not 0.03. At the far end of the shift's interval the per-question bound would be 0.044. Shifting every primary gain by the largest audited shift, 0.011, keeps 53 of 54 per-question bounds below 0.04 and all below 0.05; with errors inflated by 2.61, 51 stay below 0.05. Table LVII gives the audit. Between 27% and 40% of held-out questions are discordant in these tests, so the shift on the discordant answers is scaled by that share. The adjudicated label agreed with rater A on 31 of the 35 items. False positives of `label_final` are 0% to 1% in each arm, and the strict rule shifts the gains by -0.011, -0.016 and 0.000, close to `label_final`. On audit 3 alone the bound came near 0.04, because it adds the uncertainty of the shift to that of the test; for the uncharged per-plan router the shifted gain would be $+0.037$, but charging its embedding lowers its gains by 0.008 to 0.022 (Table XXXII).

For IRCoT the risk runs the other way, since a judge could favour its longer answers. IRCoT and its hull plan send similar shares to the judge (for example, 0.100 against 0.108 for Qwen2.5 7B on 2Wiki). On the 50 audited questions of Qwen3 14B on 2Wiki, 44% of all questions in that test, the judge does lean towards IRCoT, by $+0.017$ $[-0.017, +0.052]$: it errs on 2% of IRCoT's answers and 6% of the hull's. Without this shift the gain is still $+0.139$, and the 2Wiki wins also hold under the strict rule.

The raters of the first two audits disagree more. Cohen's kappa is 0.47 on audit 1 and 0.70 on audit 2, and Gwet's AC1 gives 0.43 and 0.69. On audit 1, rater 1 marks 39% of answers correct and rater 2 marks 66%. `label_final` agrees with rater 1 on 0.87 of items and with rater 2 on 0.69; the lenient rule agrees with rater 2 on 0.76. Rater 1 is a coauthor who labelled blind on a blank sheet; raters 2 and 3 are not authors, and the check against each rater alone was fixed before adjudication. Since every rule gives the same verdict, the conclusion does not depend on which rater is right.

TABLE LVII: Human audit of the answers on which a policy and its hull disagree (two raters who are not authors, adjudicated): error of label_final in each arm and the bias it puts on the gain.

Test	Questions	Discordant share	label_final error, policy	label_final error, hull	Bias on the gain [95% CI]
Confidence cascade, Qwen2.5 7B, single-step	100	0.27	0.060	0.040	−0.011 [−0.024, 0.000]
Per-plan LR, Qwen3 32B, single-step	100	0.40	0.040	0.040	−0.008 [−0.032, +0.012]
Reader cascade, 7B to Qwen3 14B, with two-hop	100	0.37	0.070	0.050	0.000 [−0.019, +0.022]

The strict rule is the worst rule against every reference except rater 1 alone on audit 1, where it beats the lenient rule. LLM judges must be checked per task [67], and these audits are that check for ours.

Table LIX gives each rule against the human majority, with its interval, kappa and error counts. The strict rule fails mostly through false negatives (62 and 52 on the two audits), and the lenient rule has the most false positives (50 and 28). Table LX gives the grading error behind J: sensitivity 0.663 and 0.735, specificity 0.939 and 0.941, and a reweighted J of 0.595 and 0.645. Before adjudication the two raters agree on 71.5% and 84.5% of items, with Krippendorff's alpha of 0.430 and 0.691. Table LXI is the full version of Table LVIII. In Table LIX, each row is one rule on one audit: accuracy against the majority with a 95% interval, kappa against the majority, and the false positives (rule says correct, majority says wrong) and false negatives. `label_final` has 6 false positives on each audit, close to the strict rule (11 and 4), while it cuts false negatives from 62 and 52 to 34 and 26. In Table LX, sensitivity is the share of answers the majority marks correct that `label_final` also marks correct, and specificity is the same for wrong answers. The four agreement measures of the raters tell the same story: 0.43 to 0.47 for the chance corrected measures on audit 1, and 0.69 to 0.70 on audit 2. In Table LXI, each rule is scored against three references: the majority, rater 2 alone and rater 1 alone. On audit 1 the best rule depends on the reference: `label_final` against rater 1 (0.870) and the lenient rule against rater 2 (0.760).

Q4. When does adaptive retrieval pay off? Within one dataset, none of the policies we tested pays off once it pays for its own signals. Across datasets there is room: on the mixed workload, knowing the dataset is worth up to 0.048 with the two-hop plans. A guess at the dataset that pays for its embedding gets part of it, and no policy adds to it (Section VI-F). The useful choice is between kinds of plans, one retrieval or two, and it depends mostly on the dataset. IRCoT decides after every step, once the evidence is in view, yet a chain with a fixed number of steps does as well (Section VI-G); a policy that stops early must tell when a further hop would fix the answer, close to the task our planners failed in Q2.

Q5. What should evaluations report? Data management learned a similar lesson: under fixed splits, no learned query optimizer consistently beat PostgreSQL [17], and a simple adaptive baseline matched or beat learned optimizers on many workloads [18]. For adaptive retrieval we suggest that every evaluation report the gain over the fixed plan hull at the policy's own mean cost, with hull plans and policy settings chosen on the same training folds; a test that counts the variance from refitting; results for several readers, with and without multi-hop plans, and on a mixed workload; the minimum detectable effect and a margin test for null results; a human audit of the grader; cost in hardware-free units next to latency; and the policy's own cost, such as a query embedding or a probe. The benchmark runs this protocol for any new policy over our plans without new reader calls. A new reader needs its full plan matrix, 50,000 to 62,000 answers, and a new retriever about 10,000 answers per reader.

VIII. Validity and Limitations

Hardware and replay. All latencies come from one GPU and one runtime per reader, so we repeat every primary test under four hardware-free costs, and the verdict does not change. A live end-to-end rerun for Qwen2.5 7B on MuSiQue and 2Wiki matched the replay wherever the retriever fit on the GPU next to the reader: the ratio of mean times is 0.95 to 1.00, and every decision is the same. Prompt building, tokenisation and decoding, which the replay omits, add 4 to 6 ms per call. Table LXII gives the cell check per retriever. On MuSiQue every retriever that fits matches its stored time within 1%; on 2Wiki BM25, SPLADE and closed-book match within 9%, and the two dense plans take 1.9 and 2.7 times their stored time because their index ran on the CPU. Table LXIII gives the policy check. The closed-book gate costs 0.94 of its replayed cost on both datasets, and the per-plan router 0.99 on MuSiQue and 1.10 on 2Wiki, where its 25 ms embedding is a larger share of a cheap policy. The confidence cascade on 2Wiki costs 1.20 of its replayed cost because its stronger plan uses a dense index on the CPU. Every live decision matched the stored one, and 98.5% of live answers were identical to the stored answers. On our 20 GB card, Stella and the reranker do not fit next to a reader, so the replay describes a system in which retrieval and reading each have the GPU; both arms use the same plan costs. The reader cascades assume that both readers stay loaded. On our card they do not fit together, and swapping the 7B reader for Qwen3 14B takes 45 s, so the replay understates their cost; they still do not beat the hull. Every reader serves one question at a time; batched serving would change the cost of long prompts and extra calls, so it would move the costs of IRCoT, the fixed chains and the cascades, which make several calls per question, more than those of single-step plans.

Scope. Our readers are open models of 7B to 32B parameters, the two largest at about 4 bits; the questions are short-

TABLE LVIII: Accuracy of each grading rule against three human references on the two audits of 200 answers. Maj.: majority of three raters (the reference used in the paper). R1: rater 1 alone (a coauthor, labelled blind). R2: rater 2 alone (not an author). Audit 1 covers Qwen2.5 7B, Llama 3.1 8B and Qwen3 14B; audit 2 covers Gemma 3 27B and Qwen3 32B.

	Audit 1			Audit 2		
Rule	Maj.	R1	R2	Maj.	R1	R2
Strict	0.635	0.685	0.520	0.720	0.750	0.675
F1 $\geq$ 0.7	0.735	0.755	0.700	0.800	0.820	0.765
Lenient	0.695	0.645	0.760	0.805	0.785	0.810
`label_final`	0.800	0.870	0.685	0.840	0.900	0.805
Share marked correct	0.51	0.39	0.66	0.49	0.44	0.56

TABLE LIX: Grading rules against the human reference (majority of three raters).

Audit	Rater kappa (two raters)	Rule	n	Accuracy vs majority of three	Kappa	FP	FN
Audit 1 (7B, Llama, Qwen3 14B)	0.469	strict	200	0.635 [0.565, 0.700]	0.274	11	62
Audit 1 (7B, Llama, Qwen3 14B)	0.469	F1 $\geq$ 0.7	200	0.735 [0.665, 0.795]	0.471	21	32
Audit 1 (7B, Llama, Qwen3 14B)	0.469	lenient	200	0.695 [0.630, 0.755]	0.388	50	11
Audit 1 (7B, Llama, Qwen3 14B)	0.469	label_final	200	0.800 [0.745, 0.850]	0.601	6	34
Audit 2 (Gemma, Qwen3 32B)	0.695	strict	200	0.720 [0.655, 0.780]	0.434	4	52
Audit 2 (Gemma, Qwen3 32B)	0.695	F1 $\geq$ 0.7	200	0.800 [0.745, 0.855]	0.598	11	29
Audit 2 (Gemma, Qwen3 32B)	0.695	lenient	200	0.805 [0.745, 0.860]	0.611	28	11
Audit 2 (Gemma, Qwen3 32B)	0.695	label_final	200	0.840 [0.785, 0.890]	0.679	6	26

TABLE LX: label_final against the human majority (J) and agreement of the two raters before adjudication.

Audit	Sensitivity	Specificity	J	J, reweighted	Raters: raw agreement	Cohen kappa	Gwet AC1	Krippendorff alpha
audit1	0.663	0.939	0.603	0.595	0.715	0.469	0.431	0.430
audit2	0.735	0.941	0.676	0.645	0.845	0.695	0.690	0.691

TABLE LXI: Grading rules against each human reference. Rater 1 is a co-author who labelled blind; rater 2 is not an author.

Audit	Rule	n	Majority of three	Rater 2 alone (not an author)	Rater 1 alone (author)
Audit 1 (7B, Llama, Qwen3 14B)	strict	200	0.635	0.520	0.685
Audit 1 (7B, Llama, Qwen3 14B)	F1 $\geq$ 0.7	200	0.735	0.700	0.755
Audit 1 (7B, Llama, Qwen3 14B)	lenient	200	0.695	0.760	0.645
Audit 1 (7B, Llama, Qwen3 14B)	label_final	200	0.800	0.685	0.870
Audit 2 (Gemma, Qwen3 32B)	strict	200	0.720	0.675	0.750
Audit 2 (Gemma, Qwen3 32B)	F1 $\geq$ 0.7	200	0.800	0.765	0.820
Audit 2 (Gemma, Qwen3 32B)	lenient	200	0.805	0.810	0.785
Audit 2 (Gemma, Qwen3 32B)	label_final	200	0.840	0.805	0.900

answer English QA; learned policies train on 400 questions per reader and dataset. With 2,000 pooled questions the per-question test detects a gain of about 0.027 with 80% power (0.038 at Holm's strictest step); our decision rule also needs the corrected test with errors inflated by 2.61, which detects about 0.048 (0.068 at the strictest step), so smaller gains would be missed, and we draw no per-dataset conclusions for the primary family. Our policies pick one plan per question; methods that decide during generation such as FLARE [4], read the model's internal states such as SeaKR [23], or train the reader such as Self-RAG [24] were not run. The label hull of Section VI-F gets each question's dataset for free; that is realistic for traffic tagged by source, and it is a reference for a policy that acts on the workload only, not a deployable baseline.

Implementations. We reimplement every policy within our plan space, so the conclusions concern these implementations, not adaptive retrieval in general. IRCoT is the only adaptive multi-step controller we test, so the finding on stopping rules covers IRCoT only. The implementation checks of Section V and the stronger variants reduce this risk.

Grading. An LLM judge decides 5% to 39% of labels. The human audits, three grading rules, a second judge and the equal exposure of the two arms (Section VII) reduce this risk. Rater agreement on audit 1 is only moderate. A coauthor rated audits 1 and 2, but every reference label equals the label of a rater who is not an author (rater 2 where the two agree,

TABLE LXII: Live end-to-end time against the stored time of the same cells, Qwen2.5 7B (median of three live runs per cell). Index: where the dense index sat with the reader loaded. Stella and the reranker did not fit next to the reader.

Data	Retriever	Cells	Live s	Stored s	Live / stored	Retrieval live /stored s	Index
MuSiQue	bge bm25 rrf	16	0.542	0.538	1.007	0.029 / 0.027	GPU
MuSiQue	bge m3	19	0.556	0.560	0.993	0.027 / 0.027	GPU
MuSiQue	bm25	18	0.450	0.451	0.998	0.001 / 0.001	no dense index
MuSiQue	closed-book	4	0.164	0.164	0.998	0.000 / 0.000	
MuSiQue	splade v3	11	0.473	0.475	0.997	0.040 / 0.045	no dense index
2Wiki	bge bm25 rrf	16	1.536	0.800	1.919	0.980 / 0.036	CPU
2Wiki	bge m3	19	1.521	0.570	2.668	0.979 / 0.030	CPU
2Wiki	bm25	18	0.470	0.481	0.978	0.004 / 0.004	no dense index
2Wiki	closed-book	4	0.176	0.185	0.950	0.000 / 0.000	
2Wiki	splade v3	11	0.587	0.638	0.919	0.124 / 0.152	no dense index

TABLE LXIII: Policies run live end-to-end against their replayed cost, Qwen2.5 7B (questions whose retrievers fit next to the reader). Per-plan LR includes its 24.6 ms embedding in the live time only.

Data	Policy	Questions	Live s	Replay s	Live / replay	Same decision
MuSiQue	Closed-book gate	200	0.191	0.202	0.944	1.00
MuSiQue	Per-plan LR	55	0.403	0.409	0.986	1.00
2Wiki	Confidence cascade	165	0.858	0.717	1.196	1.00
2Wiki	Closed-book gate	116	0.186	0.197	0.942	1.00
2Wiki	Per-plan LR	200	0.259	0.235	1.103	1.00

rater 3 where they differ); the raters of audits 3 and 4 are not authors.

Analyses added after the main results. Every check beyond the primary tests came after the main results: the margin test, the corrected test and its inflation factor, the mixed workload, the outside judge, audits 3 and 4, matching at tail latency, the split halves, Adaptive RAG as published and the fixed step chains. None changes the verdict of a primary test, and on audited labels the confidence cascade excludes a gain of 0.04 but not 0.03.

Reproducibility. The code, graded answers, question IDs, corpus and fold construction, audit files and library versions are public, and every analysis reruns from the stored answers on a CPU with fixed seeds.

IX. Conclusion

We asked how much a cost-matched baseline changes the verdict on adaptive retrieval. On the same answers, the protocol common in prior work credits five primary policies with 64 of 100 wins; the best random mix of two fixed plans at the same cost leaves none and explains their wins over single plans. Adaptive RAG as published beats Always RAG by 0.070 to 0.091 and loses to the mix by 0.023 to 0.061. With hulls and policies fitted per dataset, gains vanish once policies pay for their own signals; on mixed traffic, routers gain mostly by recognising the dataset and none meets our decision rule; IRCoT's wins do not depend on its stopping rule. These findings hold for open readers of 7B to 32B parameters served one question at a time, short-answer English questions and 400 training questions per reader and dataset; new policies should face the same hull, with their own costs charged.

AI-Generated Content Acknowledgement

Claude Opus 5.5 was used exclusively for the purpose of polishing and refining the linguistic style of certain sentences to improve readability and fluency of every section. The authors ran every experiment on their own hardware, checked every number, table and figure against the result files, and take responsibility for the content. Experimental Data and Results: All code, figures, findings, and interpretations thereof were generated, designed, collected, and analyzed by the authors. Two local language models are part of the method: Qwen3 14B and Mistral Small 3.1 24B grade the ambiguous answers (Section IV). Human raters did the grading audits without AI help.

References

[1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 9802–9822.

[3] Y. Wang, P. Li, M. Sun, and Y. Liu, "Self-knowledge guided retrieval augmentation for large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 10 303–10 315.

[4] Z. Jiang *et al.*, "Active retrieval augmented generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 7969–7992.

[5] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 7036–7050.

[6] C. Taguchi, S. Maekawa, and N. Bhutani, "Efficient context selection for long-context QA: No tuning, no iteration, just adaptive-k," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 20 105–20 130.

[7] J. Zhang, G. Li, and J. Su, "SAGE: A framework of precise retrieval for RAG," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 2025, pp. 1388–1401.

[8] Z. Wang, H. Yuan, W. Dong, G. Cong, and F. Li, "CARROT: A learned cost-constrained retrieval optimization system for RAG," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE, 2026, pp. 2150–2163.

[9] Y. Liu *et al.*, "SPARQ: A cost-efficient framework for offline table question answering via adaptive routing," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE, 2026.

[10] V. Moskvoretskii *et al.*, "Adaptive retrieval without self-knowledge? Bringing uncertainty back home," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 6355–6384.

[11] F. Provost and T. Fawcett, "Robust classification for imprecise environments," *Machine Learning*, vol. 42, no. 3, pp. 203–231, 2001.

[12] D. Ding *et al.*, "Hybrid LLM: Cost-efficient and quality-aware query routing," in *The Twelfth International Conference on Learning Representations*, 2024.

[13] I. Ong *et al.*, "RouteLLM: Learning to route LLMs from preference data," in *The Thirteenth International Conference on Learning Representations*, 2025.

[14] T. G. Armstrong, A. Moffat, W. Webber, and J. Zobel, "Improvements that don't add up: Ad-hoc retrieval results since 1998," in *Proceedings of the 18th ACM Conference on Information and Knowledge Management*, 2009, pp. 601–610.

[15] W. Yang, K. Lu, P. Yang, and J. Lin, "Critically examining the "neural hype": Weak baselines and the additivity of effectiveness gains from neural ranking models," in *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2019, pp. 1129–1132.

[16] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are we really making much progress? A worrying analysis of recent neural recommendation approaches," in *Proceedings of the 13th ACM Conference on Recommender Systems*, 2019, pp. 101–109.

[17] C. Lehmann, P. Sulimov, and K. Stockinger, "Is your learned query optimizer behaving as you expect? A machine learning perspective," *Proceedings of the VLDB Endowment*, vol. 17, no. 7, pp. 1565–1577, 2024.

[18] Y. Zhang, Y. Chronis, J. M. Patel, and T. Rekatsinas, "Simple adaptive query processing vs. learned query optimizers: Observations and analysis," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 2962–2975, 2023.

[19] J. Jin *et al.*, "FlashRAG: A modular toolkit for efficient retrieval-augmented generation research," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 737–740.

[20] S. Li *et al.*, "RAGPerf: An end-to-end benchmarking framework for retrieval-augmented generation systems," *Proceedings of the VLDB Endowment*, vol. 19, no. 11, pp. 3689–3703, 2026.

[21] Q. Zhang *et al.*, "XRAG: eXamining the core–benchmarking foundational components in advanced retrieval-augmented generation," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE, 2026, pp. 2697–2710.

[22] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 10 014–10 037.

[23] Z. Yao *et al.*, "SeaKR: Self-aware knowledge retrieval for adaptive retrieval augmented generation," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 27 022–27 043.

[24] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *The Twelfth International Conference on Learning Representations*, 2024.

[25] M. Xiong *et al.*, "Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs," in *The Twelfth International Conference on Learning Representations*, 2024.

[26] M. Marina *et al.*, "LLM-independent adaptive RAG: Let the question speak for itself," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 8697–8709.

[27] W. Su, Y. Tang, Q. Ai, Z. Wu, and Y. Liu, "DRAGIN: Dynamic retrieval augmented generation based on the real-time information needs of large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 12 991–13 013.

[28] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[29] B. Jin, J. Yoon, J. Han, and S. Ö. Arık, "Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG," in *The Thirteenth International Conference on Learning Representations*, 2025.

[30] H. Joren, J. Zhang, C.-S. Ferng, D.-C. Juan, A. Taly, and C. Rashtchian, "Sufficient context: A new lens on retrieval augmented generation systems," in *The Thirteenth International Conference on Learning Representations*, 2025.

[31] F. Cuconasu *et al.*, "The power of noise: Redefining retrieval for RAG systems," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 719–729.

[32] M. Mazuryk, F. Dolmans, L. Gehringer, I. Klaric, J.-H. Ju, and M. Aliannejadi, "The powerless noise: How experimental settings shape the reported power of noise," in *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2026, pp. 2973–2982.

[33] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *Transactions on Machine Learning Research*, 2024.

[34] P. Aggarwal *et al.*, "AutoMix: Automatically mixing language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 131 000–131 034.

[35] W. Jiang, S. Subramanian, C. Graves, G. Alonso, A. Yazdanbakhsh, and V. Dadu, "RAGO: Systematic performance optimization for retrieval-augmented generation serving," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 974–989.

[36] W. Jiang, M. Zeller, R. Waleffe, T. Hoefler, and G. Alonso, "Chameleon: A heterogeneous and disaggregated accelerator system for retrieval-augmented language models," *Proceedings of the VLDB Endowment*, vol. 18, no. 1, pp. 42–52, 2024.

[37] S. Ray *et al.*, "METIS: Fast quality-aware RAG systems with configuration adaptation," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025, pp. 606–622.

[38] P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, "Automated algorithm selection: Survey and perspectives," *Evolutionary Computation*, vol. 27, no. 1, pp. 3–45, 2019.

[39] C. Cameron, H. H. Hoos, and K. Leyton-Brown, "Bias in algorithm portfolio performance evaluation," in *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, 2016, pp. 712–719.

[40] M. Dehghani, Y. Tay, A. Arnab, L. Beyer, and A. Vaswani, "The efficiency misnomer," in *The Tenth International Conference on Learning Representations*, 2022.

[41] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[42] Z. Chen *et al.*, "BrowseComp-Plus: A fair and disentangled evaluation benchmark for deep search agents," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026, pp. 22 349–22 370.

[43] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015.

[44] X. Wang, C. Qu, W. Wu, J. Wang, and Q. Zhou, "Are we ready for learned cardinality estimation?" *Proceedings of the VLDB Endowment*, vol. 14, no. 9, pp. 1640–1654, 2021.

[45] Y. Han *et al.*, "Cardinality estimation in DBMS: A comprehensive benchmark evaluation," *Proceedings of the VLDB Endowment*, vol. 15, no. 4, pp. 752–765, 2022.

[46] P. Mu, A. C. Carniel, A. Barbalace, and A. Shaikhha, "[Experiment, Analysis, and Benchmark] Systematic evaluation of plan-based adaptive

query processing," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE, 2026, pp. 3501–3514.

[47] G. Liu, R. Borovica-Gajic, H. Lan, and Z. Bao, "Benchmarking RL-enhanced spatial indices against traditional, advanced, and learned counterparts," in *2026 IEEE 42nd International Conference on Data Engineering (ICDE)*. IEEE, 2026, pp. 586–599.

[48] N. Reimers and I. Gurevych, "Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2017, pp. 338–348.

[49] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018, pp. 1383–1392.

[50] J. Urbano, H. Lima, and A. Hanjalic, "Statistical significance testing in information retrieval: An empirical analysis of type I, type II and type III errors," in *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2019, pp. 505–514.

[51] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky, "With little power comes great responsibility," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 9263–9274.

[52] J. E. Smith and R. L. Winkler, "The optimizer's curse: Skepticism and postdecision surprise in decision analysis," *Management Science*, vol. 52, no. 3, pp. 311–322, 2006.

[53] T. Kwiatkowski *et al.*, "Natural Questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019.

[54] K. Lee, M.-W. Chang, and K. Toutanova, "Latent retrieval for weakly supervised open domain question answering," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 6086–6096.

[55] Z. Yang *et al.*, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 2369–2380.

[56] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proceedings of the 28th International Conference on Computational Linguistics*, 2020, pp. 6609–6625.

[57] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[58] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[59] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," in *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2288–2292.

[60] T. Formal, C. Lassance, B. Piwowarski, and S. Clinchant, "From distillation to hard negative sampling: Making sparse neural IR models more effective," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022, pp. 2353–2359.

[61] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 2318–2335.

[62] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[63] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "GPT3.int8(): 8-bit matrix multiplication for Transformers at scale," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 30 318–30 332.

[64] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 2383–2392.

[65] E. Kamalloo, N. Dziri, C. Clarke, and D. Rafiei, "Evaluating open-domain question answering in the era of large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 5591–5606.

[66] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 46 595–46 623.

[67] A. Bavaresco *et al.*, "LLMs instead of human judges? A large scale empirical study across 20 NLP evaluation tasks," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2025, pp. 238–255.

[68] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An automated evaluation framework for retrieval-augmented generation systems," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 338–354.

[69] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[70] K. L. Gwet, "Computing inter-rater reliability and its variance in the presence of high agreement," *British Journal of Mathematical and Statistical Psychology*, vol. 61, no. 1, pp. 29–48, 2008.

[71] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[72] H. W. Chung *et al.*, "Scaling instruction-finetuned language models," *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.

[73] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[74] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.

[75] R. R. Bouckaert and E. Frank, "Evaluating the replicability of significance tests for comparing learning algorithms," in *Advances in Knowledge Discovery and Data Mining (PAKDD)*, ser. Lecture Notes in Computer Science, vol. 3056, 2004, pp. 3–12.

[76] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[77] D. Lakens, "Equivalence tests: A practical primer for t tests, correlations, and meta-analyses," *Social Psychological and Personality Science*, vol. 8, no. 4, pp. 355–362, 2017.